

se puede abstraer de este modo

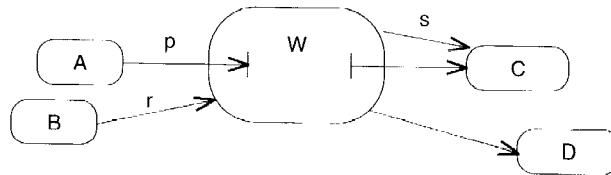


Figura 13.181 Transición abreviada

transición compleja

Transición con más de un estado origen y/o más de un estado destino. Representa una respuesta a un evento que causa un cambio en la cantidad de concurrencia. Es una sincronización de control, una división del control, o ambas, dependiendo del número de orígenes y destinos.

Véase también bifurcación, división, estado compuesto, reunificación, unión.

Semántica

A un alto nivel, un sistema pasa por una serie de estados, pero la visión monolítica de un sistema que tiene un único estado es demasiado restrictiva para grandes sistemas en los que interviene la distribución y la concurrencia. Un sistema puede estar en muchos estados simultáneamente. El conjunto de estados activos se denomina configuración del estado activo. Si está activo un estado anidado, el estado que lo contiene también está activo. Si el objeto permite concurrencia, entonces puede haber más de un subestado concurrente activo.

En muchos casos la actividad de un sistema puede modelarse como un conjunto de hilos de control que evolucionan independientemente unos de otros o que interactúan de forma limitada. Cada transición afecta, al menos, a unos pocos estados de la configuración del estado activo. Cuando se dispara una transición, los estados no afectados por la misma permanecen activos. El progreso de los hilos en un momento determinado se puede capturar como un subconjunto de estados dentro de la configuración del estado activo, de forma que exista un subconjunto por cada hilo. Cada conjunto de estados evoluciona de forma independiente en respuesta a eventos. Si el número de estados activos es constante, el modelo de estados no es sino una colección fija de máquinas de estados que interactúan. Sin embargo, en general el número de estados (y por tanto el número de hilos de control) puede variar a lo largo del tiempo. Se puede pasar de un estado a dos o más estados concurrentes (división del control), y se puede pasar de dos o más estados concurrentes a un único estado (unión de control). La máquina de estados del sistema controla el número de estados concurrentes y su evolución.

Una transición compleja es una transición hacia o desde un conjunto de subestados. Una transición compleja tiene más de un estado origen y/o más de un estado destino. Si tiene múltiples estados origen representa una unión de control, mientras que si tiene múltiples estados destino representa una división del control; si cuenta con múltiples estados origen y destino representa una sincronización de hilos paralelos.

Si una transición compleja tiene múltiples estados origen, todos ellos deben estar activos antes de que la transición sea candidata a ser disparada, siendo irrelevante el orden en que llegan a estar activos. Si todos los estados origen están activos y se produce el evento, se dispara la transición y puede ejecutarse si su condición de guarda es verdadera. Cada transición se dispara a partir de un único evento, incluso aunque haya múltiples estados origen, pues en UML no hay ocurrencia simultánea de eventos: cada evento debe disparar una transición distinta, aunque a los estados resultantes puede suceder una unión de estados.

Si una transición compleja con múltiples estados carece de un evento que la dispare (es decir, si es una transición de finalización) entonces se dispara cuando todos sus estados origen explícitos estén activos. Se lleva a cabo si en ese momento su condición de guarda es verdadera.

Cuando se lleva a cabo una transición compleja, todos los estados origen y todos sus iguales dentro del mismo estado compuesto dejan de estar activos y pasan a estarlo todos los estados destino y sus iguales.

En situaciones más complicadas, puede expandirse la condición de guarda para permitir la ejecución de la transición cuando esté activo algún subconjunto de los estados.

Ejemplo

La Figura 13.182 muestra un típico estado compuesto concurrente con transiciones complejas de entrada y de salida. La Figura 13.183 muestra una típica historia de ejecución de esta máquina (los estados activos se representan mediante letra en azul). La historia muestra la variación en el número de estados activos a lo largo del tiempo.

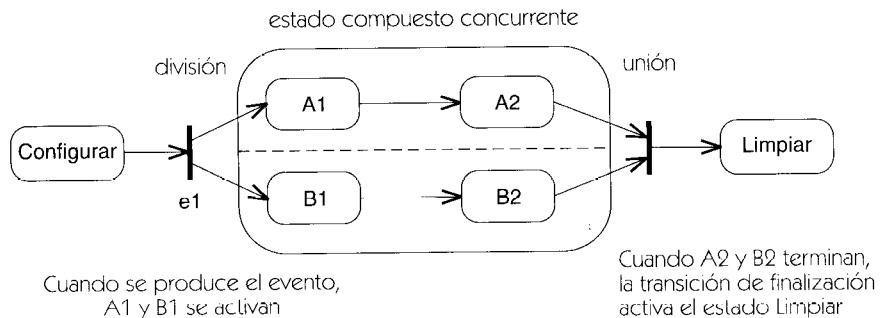


Figura 13.182 División y unión

Estados concurrentes

A menos que una máquina de estados haya sido cuidadosamente estructurada, un conjunto de transiciones complejas puede derivar en inconsistencias, incluyendo interbloques, ocupa-

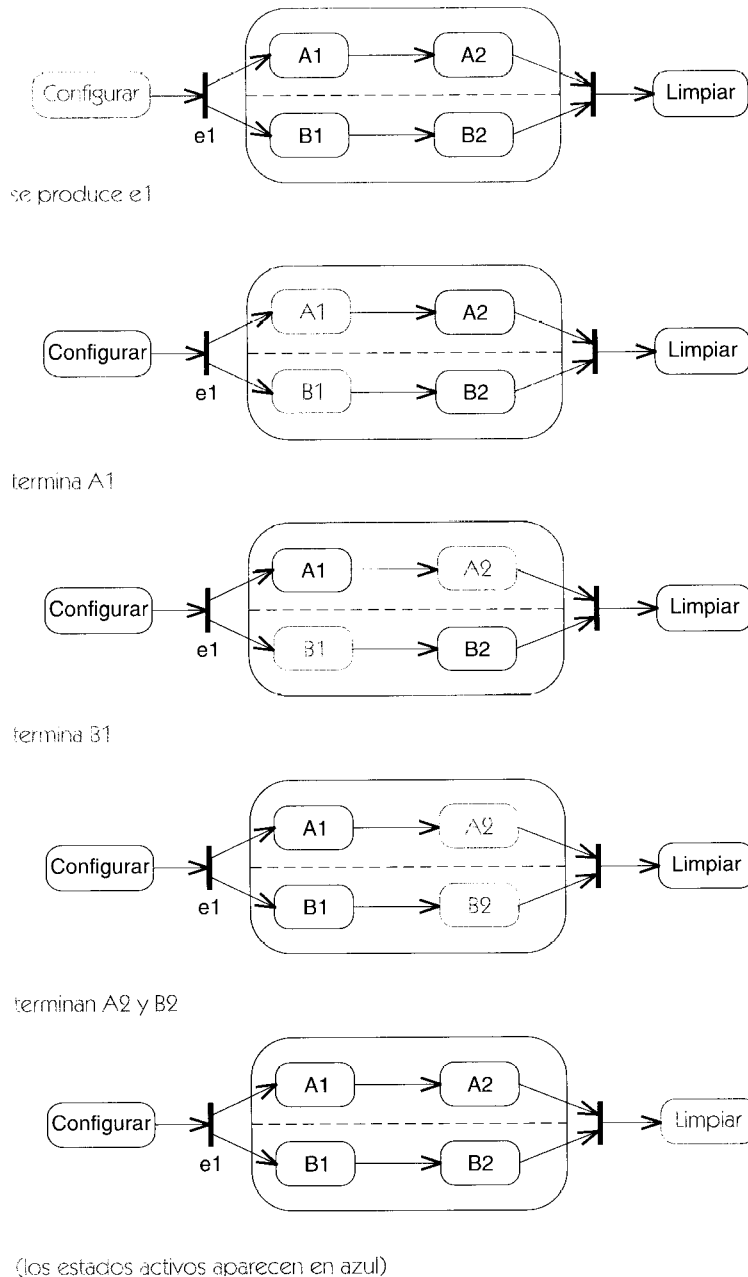


Figura 13.183 Historia de estados activos en una máquina de estados concurrente

ción múltiple de un estado, y otros problemas. El problema ha sido estudiado en profundidad a la luz de la teoría de redes de Petri, y la solución más habitual es imponer reglas de formación correcta a la máquina de estados para evitar el riesgo de inconsistencias, que son algo así como las reglas de “programación estructurada” para las máquinas de estados. Hay numerosos modelos, cada uno con sus ventajas e inconvenientes. Las reglas adoptadas por UML requieren que la máquina de estados se descomponga en estados más pequeños utilizando una especie de

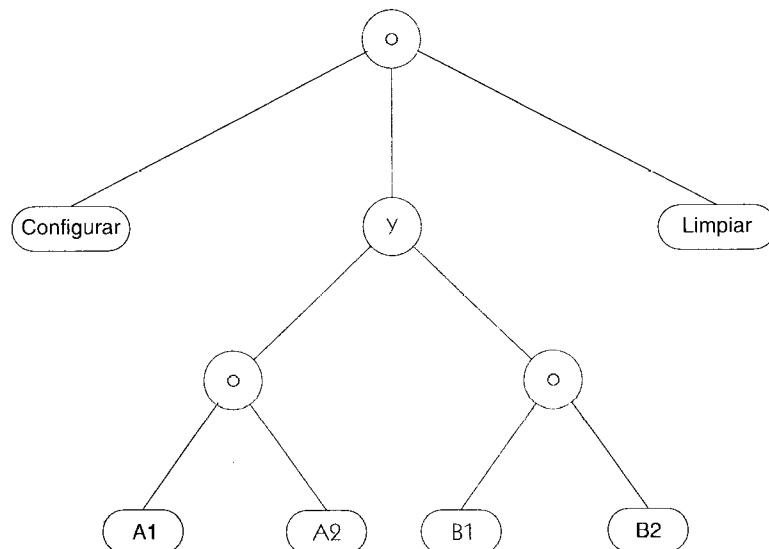
árbol Y/O. La ventaja es que una estructura bien anidada resulta sencilla de establecer, de mantener y de comprender. La desventaja es que hay ciertas configuraciones que están prohibidas aunque tienen significado. Haciendo balance, podemos decir que es algo similar a las soluciones de compromiso tomadas en referencia a las cláusulas “goto” en la programación estructurada.

Un estado complejo puede descomponerse en un conjunto de subestados mutuamente exclusivos (descomposición “O”) o en un conjunto de subestados concurrentes (descomposición “Y”). La estructura es recursiva. Generalmente, las capas “Y” se alternan con las capas “O”. Una capa “Y” representa una descomposición concurrente —todos los subestados están activos concurrentemente—, mientras que una capa “O” representa una descomposición secuencial —sólo hay un estado activo al mismo tiempo—. Se puede obtener un conjunto legítimo de subestados concurrentes expandiendo recursivamente los nodos del árbol empezando por la raíz, remplazando todos los estados “O” por todos sus hijos y remplazando todos los estados “Y” por todos sus hijos. El resultado obtenido se corresponde con la estructura anidada de los diagramas de estados.

Ejemplo

La Figura 13.184 muestra un árbol Y/O de estados correspondiente a la máquina de estados de la Figura 13.182. Se muestra un típico conjunto de estados activos en azul que se corresponde con el tercer paso de la Figura 13.183.

Si una transición entra en una región concurrente, entra en todos sus subestados, mientras que si entra en una región secuencial, entra exactamente en un subestado. El estado activo de una región secuencial puede cambiar, pero en una región concurrente todos los subestados concurrentes permanecen activos durante el tiempo en que la región está activa, aunque cada subestado concurrente se descompone a su vez en una región secuencial.



(El conjunto típico de estados activos aparece en azul)

Figura 13.184 Árbol Y/O de estados anidados

Por consiguiente, una transición simple (una que tiene sólo una entrada y una salida) debe conectar dos estados de la misma región secuencial o separados sólo por niveles "O". Una transición compleja debe conectar todos los subestados de una región concurrente con un estado externo a la región concurrente (se omiten deliberadamente casos más complicados, pero seguirían los mismos principios expuestos más arriba). En otras palabras, una transición de entrada en una región concurrente debe entrar en cada subestado, y una transición de salida de una región concurrente debe salir de todos los subestados.

Existe una representación abreviada: si una transición compleja entra en una región concurrente pero omite una o más de sus subregiones, existe una transición implícita al estado inicial de cada subregión omitida. Si alguna subregión no tiene estado inicial, el modelo está mal formado. Si una transición compleja sale de una región concurrente, existe una transición implícita para cada subregión omitida. Si se dispara la transición, finaliza toda actividad dentro de la subregión —es decir, representa una salida forzada—. Una transición puede conectarse con la propia región concurrente, lo que implica una transición al estado inicial de cada subregión —una situación muy común de modelado—. De forma parecida, una transición desde una región concurrente que engloba diversas subregiones provoca una salida forzada de cada una de las subregiones (si tiene un evento disparador) o una espera hasta que finaliza cada subregión (si no existen disparadores).

Las reglas sobre transiciones complejas aseguran que no pueda haber combinaciones de estados carentes de significado activas simultáneamente. Un conjunto de subestados concurrentes es una partición del estado compuesto que las engloba, donde o bien están todos activos o bien no lo está ninguno.

Hilo condicional

En un grafo de actividades, un segmento que sale de una división puede tener una condición de guarda. Esto es un hilo condicional. Cuando se dispara la transición, el hilo que comienza por el segmento que contiene la condición de guarda se inicia sólo si se satisface la condición de guarda. Los segmentos sin guardas siempre se inician cuando se dispara la transición. La concurrencia en un grafo de actividades debe estar bien anidada: cada división debe corresponderse con una unión posterior. Cuando un hilo condicional no puede iniciarse debido a que su condición de guarda se evalúa a falso, la actividad del segmento de entrada correspondiente en la unión posterior se considera completa, esto es, la transición no espera un flujo de control de dicho hilo condicional. Si ningún hilo de una división puede iniciarse, el control pasa inmediatamente a la unión.

Un hilo condicional es equivalente a un grafo con una bifurcación y una reunificación rodeando la parte condicional del grafo de actividades.

Ejemplo

La Figura 13.185 muestra un grafo de actividades con dos hilos condicionales, que representa el procedimiento de admisión que sigue una línea aérea. Antes de que nada pueda ocurrir, el cliente debe presentar el billete. A partir de ese momento hay tres hilos concurrentes, dos de los cuales son condicionales, ya que la asignación de asiento se realiza siempre. La comprobación del equipaje sólo se realiza si el pasajero tiene equipaje, y el pasaporte sólo se comprueba si el vuelo es internacional. Cuando se completan todos los hilos el control se une en un único hilo

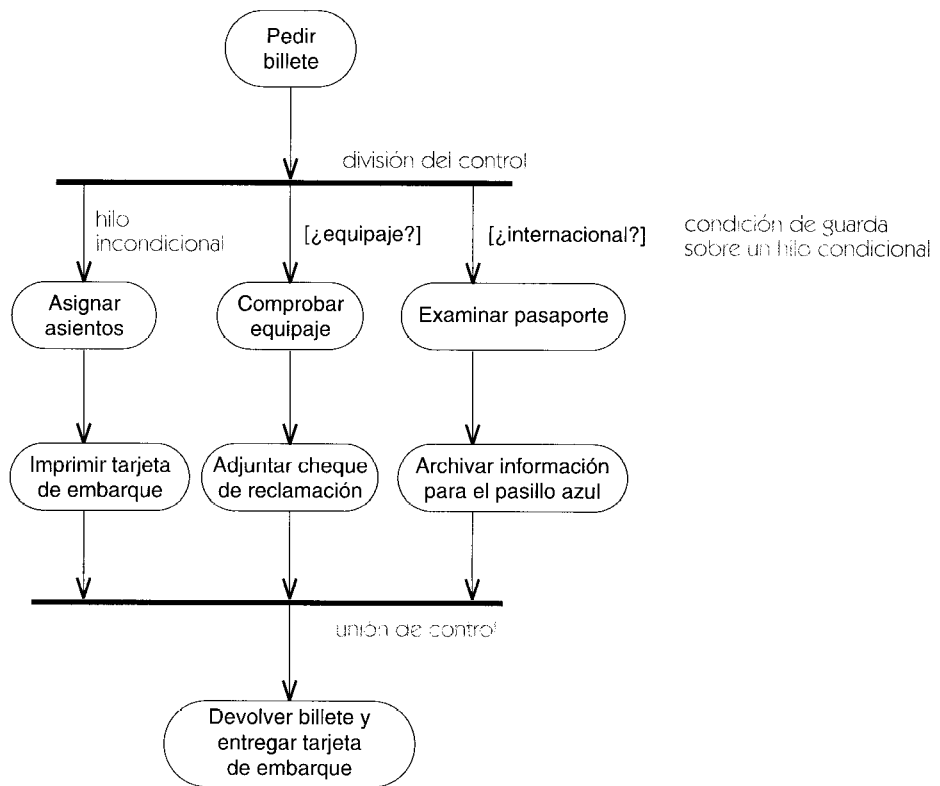


Figura 13.185 Hilos condicionales

en el cual se devuelve el billete al pasajero y se le entrega la tarjeta de embarque. Si no se inicia algún hilo condicional, se considera completado en la unión posterior.

Notación

Una transición compleja se representa mediante una pequeña barra gruesa (una barra de sincronización, que puede representar sincronización, división o ambas cosas). La barra puede tener una o más flechas de transición (dibujadas con línea continua) que van de los estados origen a la barra, y una o más flechas de transición desde la barra a los estados destino. Se puede mostrar una etiqueta de transición cerca de la barra para describir el evento disparador, la condición de guarda o la descripción de las acciones que se producen bajo la transición. Las flechas individuales no tienen sus propias cadenas de transición, ya que son meras partes de la transición global.

Ejemplo

La Figura 13.186 muestra la máquina de estados de la Figura 13.182 a la que se ha añadido una transición de salida adicional. También muestra una división implícita desde el estado **Configurar** hacia los estados iniciales de cada subregión y una unión implícita desde los estados finales de cada subregión hacia el estado **Limpiar**.

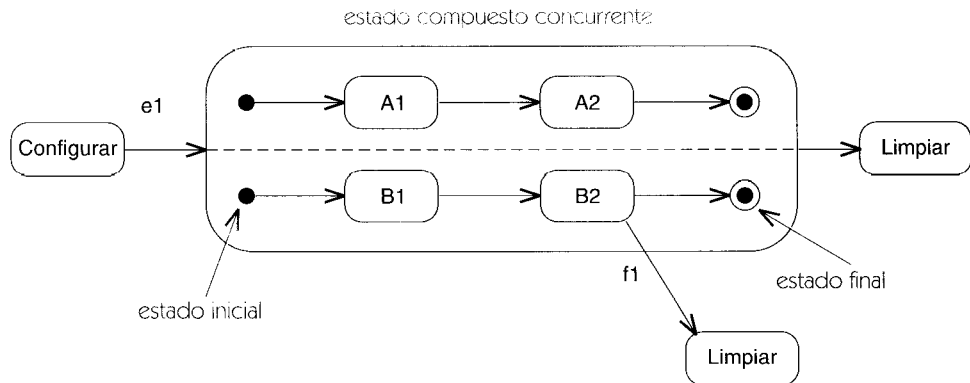


Figura 13.186 Transiciones complejas (división, unión)

Si se produce el evento **f1** cuando está activo el estado **B2**, se produce una transición al estado **Limpiar**. Esta transición es una unión implícita, ya que produce la finalización del estado **A2** a la vez que finaliza el estado **B2**.

transición de finalización

Transición que carece de un evento explícito de disparo y que se dispara por la finalización de la actividad del estado origen.

Véase también actividad, disparador, transición.

Semántica

Una transición de finalización se representa mediante una transición que no tiene un evento explícito de disparo. La transición se dispara implícitamente cuando su estado origen ha completado toda su actividad (incluyendo sus estados anidados). En un estado compuesto, la finalización de la actividad supone alcanzar el estado final. Si un estado no tiene actividad interna ni estados anidados, la transición de finalización se dispara inmediatamente después de la ejecución de la acción de entrada y de la acción de salida sin la intervención de otros eventos.

Si un estado tiene estados anidados o actividad interna pero carece de transiciones de salida disparadas por eventos, no está garantizada la ejecución de la transición de salida. Un evento podría disparar una transición sobre uno de los estados anidados mientras el estado que lo engloba está a la espera de completar su actividad, en cuyo caso la transición de finalización no se llevaría a cabo.

Una transición de finalización puede tener una condición de guarda y una acción. Normalmente no es deseable la existencia de una única transición de finalización con condición de guarda ya que si dicha condición es falsa nunca se dispara la transición (debido a que el disparador implícito sucede sólo una vez). Algunas veces esto puede ser útil para representar algún tipo de fallo, siempre que alguna transición saque al objeto de esa situación. Lo más común es que el conjunto de transiciones de finalización con condiciones de guarda tengan condiciones que cubran todas las posibilidades de forma que siempre se ejecute alguna de ellas cuando finalice el estado.

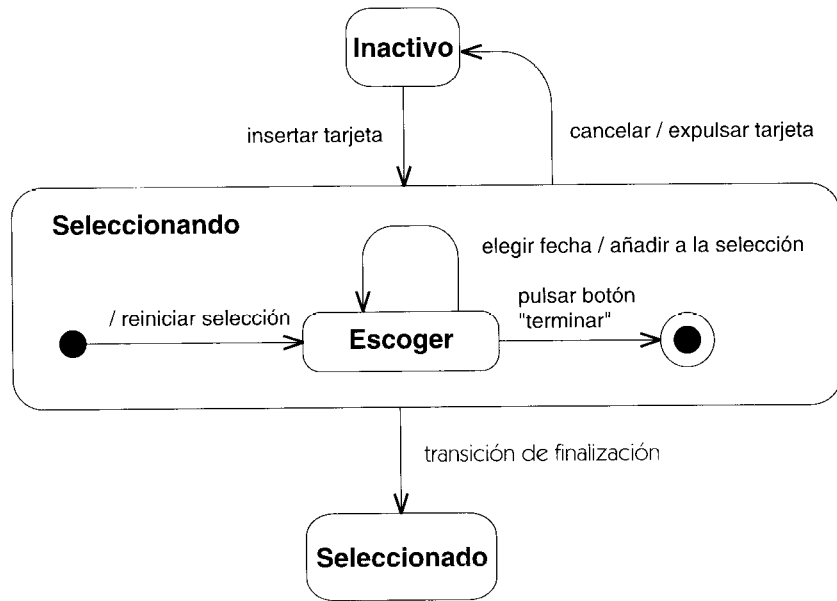


Figura 13.187 Transición de finalización

Las transiciones de finalización también se emplean para conectar estados iniciales y estados históricos con sus estados sucesores, ya que estos pseudoestados no pueden seguir activos después de la finalización de la actividad.

Ejemplo

La Figura 13.187 muestra un fragmento de una máquina de estados para una aplicación de un dispensador de entradas. El estado **Seleccionando** permanece activo durante todo el tiempo en que el cliente se encuentra eligiendo fechas. Cuando el cliente pulsa el botón “aceptar”, el estado **Seleccionando** alcanza su estado final, lo que dispara su transición de finalización y lleva a la máquina de estados al estado **Seleccionado**.

transición (fase)

Cuarta fase del proceso de desarrollo del software, durante la cual el sistema implementado se configura para su ejecución en un contexto del mundo real. Durante esta fase, se completa la vista de despliegue, junto con cualquiera de las vistas restantes que pudieran no haber finalizado en las fases anteriores.

Véase proceso de desarrollo.

transición interna

Se trata de una transición asociada a un estado que posee una acción pero no implica un cambio de estado.

Véase también máquina de estados.

Semántica

La transición interna permite que un evento dé lugar a una acción sin que haya un cambio de estado. Una transición interna posee un estado de origen pero no un estado destino. Si se dispara, se ejecuta la acción pero no cambia el estado aun cuando la transición interna esté conectada a un estado que englobe al estado actual y herede (la transición) de él. Por tanto, no se ejecuta ninguna acción de entrada ni de salida. En este aspecto, difiere de una autotransición, que da lugar a la salida de los estados anidados y a la ejecución de las acciones de salida y de entrada.

Notación

La transición interna se representa mediante una entrada de texto en el compartimento del estado destinado a la transición interna. La entrada posee la misma sintaxis que el rótulo de texto correspondiente a una transición externa. Como no hay un estado destino, no hay necesidad de asociarla a una flecha.

nombre de evento / expresión de acción

Los nombres de evento **entry**, **exit**, **do** e **include** son palabras reservadas y no se pueden emplear como nombres de evento. Estas palabras reservadas se emplean para declarar una acción de entrada, una acción de salida, la ejecución de una actividad o la ejecución de una submáquina, respectivamente. Por uniformidad, estas acciones especiales hacen uso de la sintaxis de transición interna para especificar la acción. Sin embargo, no son transiciones internas y las palabras reservadas no son nombres de eventos.

La Figura 13.188 muestra la notación.

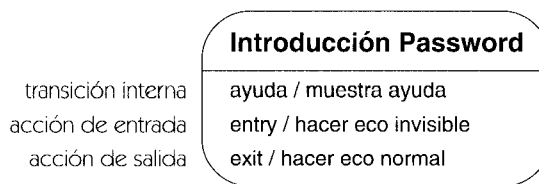


Figura 13.188 Sintaxis de transición interna

Discusión

Se puede pensar que una transición interna es una “interrupción” que da lugar a una acción sin afectar al estado actual; por tanto no invoca las acciones de salida ni las de entrada. La asociación de una transición interna a un estado compuesto es una buena forma de modelar una acción que debe suceder para un cierto número de estados pero no debe cambiar el estado activo —por ejemplo, para visualizar un mensaje de ayuda o para contar el número de apariciones de un cierto evento—. No es la forma correcta de modelar una salida por aborto o una excepción. Éstas deberían modelarse mediante transiciones a un nuevo estado, por cuanto su aparición invalida el estado actual.

transición simple

Transición con un único estado origen y un único estado destino. Representa la respuesta a un evento con un cambio de estado dentro de una región de estados mutuamente excluyentes. La cantidad de concurrencia no cambia cuando se ejecuta.

transición sin disparador

Transición que carece de un evento disparador explícito. Cuando sale de un estado normal, representa una transición de finalización, esto es, una transición que es disparada por la finalización de una actividad más que por un evento explícito. Cuando sale de un pseudoestado, representa un segmento de transición que se recorre automáticamente cuando el segmento precedente ha concluido su acción. Las transiciones sin disparador se utilizan para conectar estados iniciales y estados de historia con sus estados blanco.

traza

Dependencia que indica un proceso de desarrollo histórico o alguna otra relación más allá del modelo entre dos elementos que representan el mismo concepto sin reglas específicas para derivar uno a partir del otro. Se trata de la clase de dependencia menos específica y posee una semántica mínima. Su mayor aplicación es como recordatorio para el pensamiento humano durante el desarrollo.

Véase dependencia, modelo.

Semántica

Una traza es una variedad de dependencia que indica una conexión entre dos elementos que representan el mismo concepto en dos niveles diferentes de significado. No representa una semántica dentro de un modelo. Más bien, representa conexiones entre elementos de semántica diferente —esto es, de elementos pertenecientes a diferentes modelos situados en distintos planos de significado—. No hay una correspondencia explícita entre los elementos. Con cierta frecuencia, representa una conexión entre dos formas de capturar un concepto en distintas fases de desarrollo. Por ejemplo, dos elementos que sean variaciones de un mismo tema podrían estar relacionados por una traza. Una traza no representa una relación entre instancias en tiempo de ejecución. Más bien, es una dependencia entre elementos del modelo en sí.

Una aplicación importante de la traza es el seguimiento de requisitos que hayan ido cambiando a lo largo del desarrollo de un sistema. Las dependencias de traza pueden relacionar elementos de dos clases diferentes de modelos (tales como un modelo de caso de uso y un modelo de diseño) o pertenecientes a dos versiones de la misma clase de modelo.

Notación

Las trazas se denotan mediante una flecha de dependencia (una flecha discontinua el origen en el elemento más moderno y la punta de flecha en el elemento más antiguo) que posee la pala-

bra clave «**trace**». Normalmente, sin embargo, los elementos están en diferentes modelos que no se visualizan simultáneamente, así que en la práctica la relación se implementaría en forma de un hipervínculo dentro de alguna herramienta.

tupla

Lista ordenada de valores. Generalmente, el término implica que existe un conjunto de listas de forma similar. (Es un término matemático estándar.)

unidad de distribución

Un conjunto de objetos o componentes que se asignan a un proceso del sistema operativo o a un procesador como grupo. Una unidad de la distribución se puede representar por un compuesto de ejecución o por un agregado. Esto es un concepto de diseño en la vista de despliegue.

unión

Denota un cierto lugar de una máquina de estados, diagrama de actividades, o diagrama de secuencia en el cual se combinan dos o más hilos o estados concurrentes para formar un único hilo o estado; es una reunión o “confluencia”. Antónimo: bifurcación.

Véase transición compleja, estado compuesto.

Semántica

Diremos que una unión es una transición con dos o más estados de origen y un único estado blanco. Si todos los estados de origen están activados y se produce el evento disparador, se ejecuta la acción de transición y se activa el estado blanco. Los estados de origen tienen que estar en regiones diferentes de un estado compuesto concurrente.

Notación

Las uniones se representan mediante una barra gruesa con dos o más flechas entrantes de transición y una flecha saliente de transición. Puede tener un rótulo de transición (condición de guarda, evento disparador y acción). La Figura 13.189 muestra una unión explícita de estados procedentes de un estado compuesto concurrente.

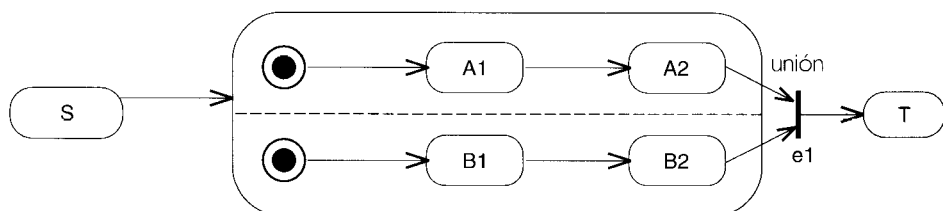


Figura 13.189 Una unión

Discusión

Véase reunificación.

use

Palabra clave para la dependencia de uso en la notación.

uso

Dependencia en la que un elemento (el cliente) requiere la presencia de otro elemento (el proveedor) para su correcto funcionamiento o implementación —generalmente es por razones de implementación.

Véase también colaboración, dependencia, enlace transitorio.

Semántica

Una dependencia de uso es una situación en la cual un elemento requiere la presencia de otro elemento para su correcta implementación o funcionamiento. Todos los elementos tienen que existir en el mismo nivel de significado —esto es, no implican un desplazamiento del nivel de abstracción o de realización (tal como una correspondencia entre una clase del nivel de análisis y una clase de nivel de implementación)—. Con cierta frecuencia las dependencias de uso implican elementos del nivel de implementación, tales como un archivo *include* de C++, lo que implica consecuencias para el compilador. Se puede estereotipar aún más el uso para indicar la naturaleza exacta de la dependencia, tal como llamar a una operación de otra clase o instanciar un objeto de otra clase.

Notación

El uso se denota mediante una flecha discontinua (dependencia) que tiene la palabra clave «**use**». La cabeza de flecha está en el elemento proveedor (independiente) y la cola se encuentra en el elemento cliente (dependiente).

Discusión

Un uso suele corresponder a un enlace transitorio, esto es, una conexión entre instancias de clases que no tiene sentido o no está presente en todo momento, sino sólo en algún contexto, tal como la ejecución de un procedimiento de subrutina. La estructura de dependencia no modela toda la información de esta situación, sino únicamente el hecho de su existencia. La estructura de colaboración proporciona la posibilidad estas relaciones con todo detalle.

Elementos estándar

call, create, instantiate, send.

uso de la tipografía

El texto puede ser distinguido con el uso de diversas tipografías y de otros marcadores gráficos.

Véase también marcador gráfico.

Discusión

Las letras cursivas se utilizan para indicar una clase abstracta, un atributo, o una operación. Otras distinciones de la tipografía están sobre todo para destacar o distinguir las partes de la notación. Se recomienda que los nombres de clasificadores y de asociaciones se muestren en negrillas y elementos subsidiarios, tales como atributos, operaciones, nombre de rol, etcétera, se muestre en tipo normal. Los nombres de compartimentos se deben mostrar en una tipografía distintiva, tal como una negrita pequeña, pero la elección se deja a la herramienta de edición. Una herramienta es también libre de utilizar las distinciones de la tipografía para destacar elementos seleccionados, para distinguir palabras y palabras claves reservadas, y para codificar características seleccionadas de un elemento, o puede permitir el uso de tales distinciones bajo control del usuario. Consideraciones similares se aplican al color, aunque su uso debe ser opcional porque muchas personas no distinguen el color.

Todos los usos mencionados son extensiones convenientes a la notación canónica descrita en este libro, la cual es suficiente para representar cualquier modelo.

utilidad

Un estereotipo de clase que agrupa variables y procedimientos globales en forma de una declaración de clase. Los atributos y operaciones de la utilidad pasan a ser variables y procedimientos globales, respectivamente. Una utilidad no es una estructura fundamental para el modelado, sino algo cómodo para la programación. No posee instancias.

Semántica

Los atributos de la utilidad son variables globales y las operaciones de la utilidad son operaciones globales. Las utilidades son innecesarias para la programación orientada a objetos, por cuanto los atributos y operaciones globales se pueden modelar mejor como miembros con alcance de clase.

Esta estructura se proporciona por compatibilidad con los lenguajes no orientados a objetos, tales como C.

Notación

Las utilidades se representan mediante un símbolo de clase con la palabra clave «**utility**» por encima de la cadena de nombre de clase. Los atributos y operaciones representan miembros globales. En este símbolo no se pueden declarar miembros con alcance de clase.

valor

Véase valor dato.

valor dato

Una instancia de un tipo de dato, un valor sin identidad.

Véase también tipo de dato, objeto.

Semántica

Un valor dato es un miembro de un dominio matemático —un valor puro—. Dos valores dato con la misma representación son indistinguibles; los valores dato no tienen ninguna identidad. Los valores dato son pasados por valor en un lenguaje de programación. No tiene ningún sentido pasarlos por referencia. No tiene sentido hablar de cambiar un valor dato; su valor está fijado permanentemente. De hecho, él es su valor. Cuando uno habla de cambiar un valor de un dato, esto significa generalmente cambiar una variable que contiene un valor de modo que contenga un nuevo valor. Pero los valores dato son invariables.

valor etiquetado

Pareja formada por un marcador y un valor, que se asocia a un elemento para almacenar alguna información. *Véase también* restricción, estereotipo. *Véase* el Capítulo 14, Elementos Estándar, para consultar una lista de marcadores predefinidos.

Semántica

Un valor etiquetado es una pareja valor-selector que se asocia a cualquier elemento (incluyendo elementos del modelo y elementos de presentación) para aportar distintos tipos de información, que suelen ser secundarios para la semántica del elemento pero posiblemente importantes para el esfuerzo global de modelado. El selector se denomina marcador; se trata de un valor de cadena. Cada marcador representa una propiedad que puede ser aplicable a una clase de elemento o a muchas clases. El nombre del marcador sólo puede aparecer una vez en cualquier elemento del modelo. El valor puede ser de distintos tipos, pero se codifica como una cadena. La interpretación del valor es una convención entre el creador del modelo y la herramienta de modelado. Se supone que los valores etiquetados se van a implementar como tablas de búsqueda indexadas por marcadores para tener un acceso eficiente.

Los valores etiquetados representan una información arbitraria expresada en forma de texto y que se suele utilizar para almacenar información relativa a la administración del proyecto, tal como el autor de un elemento, el estado de comprobación o la importancia del elemento para el sistema final (los marcadores podrían ser **autor**, **estado** e **importancia**).

Los valores etiquetados representan una modesta extensión de los metaatributos de las metaclases de UML. No es un mecanismo de extensión completamente general pero se puede utilizar para añadir información a las clases existentes del metamodelo para bien de las herramientas finales, tales como generadores de código, creadores de informes y simuladores. Para evitar la confusión, los marcadores deben ser distintos de los metaatributos existentes de los elementos del modelo al que se aplican. Esta comprobación se puede facilitar mediante una herramienta de modelado.

Ciertos marcadores están predefinidos en el UML; otros pueden ser definidos por el usuario. Los valores etiquetados son un mecanismo de extensión que permite asociar información arbitraria a los modelos.

Notación

Todos los valores etiquetados se muestran en la forma:

`marcador = valor`

en donde `marcador` es el nombre del marcador y `valor` es un valor literal. Los valores etiquetados se pueden incluir junto con otras palabras clave de propiedades en una lista de propiedades separadas por comas y encerrada entre llaves.

Se puede declarar una palabra clave de tal modo que haga las veces de un marcador con un cierto valor. En tal caso, la palabra clave se puede emplear en solitario. La ausencia del marcador se trata como equivalente a uno de los demás valores válidos del marcador.

`marcador`

Ejemplo

`{autor = Héctor, estado = comprobado, requisito = 3.563.2a, suprimir}`

Discusión

Casi todos los programas de edición de modelos ofrecen capacidades básicas para definir, visualizar y buscar valores etiquetados como cadenas, pero no los utilizan para extender la semántica de UML. Sin embargo, las herramientas finales, tales como los generadores de código, creadores de informes y cosas similares, pueden leer los valores etiquetados para alterar su semántica de manera flexible. Observe que las listas de valores etiquetados son una idea muy antigua; por ejemplo, las listas de propiedades del lenguaje Lisp.

Los valores etiquetados son una forma de asociar a los modelos información no semántica de administración y seguimiento del proyecto. Por ejemplo, el marcador `autor` podría contener el autor del elemento y el marcador **estado** podría contener el estado de desarrollo, tal como **incompleto**, **probado**, **defectuoso** y **completo**.

Además, los valores etiquetados son una forma de asociar condiciones dependientes del lenguaje de implementación a los modelos de UML sin incorporar a UML los detalles del lenguaje. Los indicadores propios de un generador de código, las pistas y los pragmas se pueden codificar como valores etiquetados sin afectar al modelo subyacente. Se pueden tener múl-

tiples conjuntos de marcadores para diferentes lenguajes en un mismo modelo. Ni el editor de modelos ni el analizador semántico tienen necesidad de comprender los marcadores —se pueden tratar como si fuesen cadenas—. Una herramienta final, tal como un generador de código, puede comprender y procesar los valores etiquetados. Por ejemplo, un valor etiquetado podría dar nombre a la clase de contenedor que se emplea para anular la implementación por defecto de una asociación con multiplicitad muchos.

Los valores etiquetados satisfacen la necesidad de muchas clases de información que es preciso asociar a los modelos, pero no están destinados a ser un mecanismo completo de extensión de metamodelos. Los marcadores forman un espacio de nombres plano que es preciso administrar empleando convenciones para evitar conflictos de nombres. No se ha previsto la posibilidad de especificar los tipos que poseen su valores. Tampoco están destinados a crear extensiones semánticas serias del lenguaje de modelado en sí. Los marcadores *vienen a ser* como atributos del metamodelo, pero *no* son atributos del metamodelo ni se han formalizado como tales.

El uso de marcadores, tal como el uso de procedimientos, en las bibliotecas de los lenguajes de programación, puede requerir un cierto período de evolución durante el cual puede haber conflictos entre los programadores. Con el tiempo, se desarrollarán estándares de utilización. UML no incluye un “registro” de marcadores ni ofrece la esperanza de que los primeros usuarios de los marcadores puedan “reservarlos” para evitar que se les den otros usos en el futuro.

valor inicial

Se trata de una expresión que especifica el valor que contiene un atributo de un objeto inmediatamente después de ser inicializado.

Véase también valor por defecto, iniciación.

Semántica

Un valor inicial es una expresión asociada a un atributo. La expresión es una cadena de texto que también denota un lenguaje para interpretar la expresión. Cuando se instancia un objeto que contiene ese atributo, se evalúa la expresión según el lenguaje indicado y el valor actual del sistema. El resultado de la evaluación se emplea para iniciar el valor del atributo en el nuevo objeto.

El valor inicial es opcional. Si está ausente, entonces la declaración del atributo no especifica el valor que contendrá en un objeto nuevo (pero quizá otra parte del modelo total proporcione esa información).

Observe que un procedimiento explícito de inicialización para un objeto (un constructor, por ejemplo) puede anular la expresión del valor inicial y reemplazar el valor del atributo.

El valor inicial de un atributo cuyo alcance sea una clase se empleará para iniciarlo al comenzar la ejecución. UML no especifica el orden relativo de inicialización de atributos que posean distintos alcances de clase.

valor no especificado

Valor que todavía no ha sido especificado por el creador del modelo.

Discusión

Un valor no especificado no es un valor en estado correcto y no puede aparecer en un modelo complejo salvo para aquellas propiedades cuyo valor es innecesario o irrelevante. Por ejemplo, la multiplicidad no puede ser desconocida; tiene que tener algún valor. La falta de todo conocimiento es equivalente a una multiplicidad de muchos. La semántica de UML, por tanto, no permite ni trata la ausencia de valores o los valores no especificados.

Hay otro sentido en que el término “no especificado” es una parte útil de un modelo inacabado. Tiene el significado siguiente: “Todavía no he pensado en este valor y he tomado nota de él, así que lo recordaré posteriormente para darle un valor”. Se trata de una indicación explícita de que el modelo está incompleto. Esto es una capacidad útil que muy bien podrían admitir las herramientas. Por su propia naturaleza, tal valor no puede aparecer en un modelo terminado y no tiene sentido definir su semántica. Una herramienta puede proporcionar automáticamente uno por defecto para los valores no especificados cuando se necesite un valor —por ejemplo, en la generación de código— pero esto no pasa de ser una cuestión de comodidad y no forma parte de la semántica. Los valores no especificados van más allá de la semántica de UML.

Análogamente, el valor por defecto de una propiedad carece de significado semántico. Las propiedades tienen un valor, simplemente. Una herramienta puede proporcionar automáticamente valores para las propiedades de los elementos recién creados. Ahora bien, esto no es más que una cierta comodidad operativa dentro de la herramienta; no forma parte de la semántica de UML. Los modelos semánticamente completos de UML no tienen valores por defecto ni valores no especificados; tienen valores, simplemente.

valor por defecto

Un valor proporcionado automáticamente como parte de un cierto lenguaje de programación o de una determinada herramienta. Los valores por defecto para las propiedades de los elementos no son parte de la semántica de UML y no aparecen en modelos.

Véase también valor inicial, parámetro, valor no especificado.

variabilidad

Propiedad que indica si puede cambiar el valor de un atributo o de un enlace.

Semántica

Esta propiedad se puede poner en un extremo de asociación o en un atributo. Si se aplica a una clase, significa que todos los atributos y enlaces de la misma la verifican (por ejemplo, un va-

lor *frozen* significa que el valor de un objeto de la clase no se puede modificar después de la inicialización). La variabilidad se representa mediante una palabra clave en una lista de propiedades con alguno de los siguientes valores:

changeable

Los valores de los atributos pueden cambiar libremente, incluyendo la adición y eliminación de valores si lo permite la multiplicidad. Los enlaces pueden cambiar libremente, y ser añadidos y eliminados respetando las restricciones y la multiplicidad. Éste es el valor por defecto si no se especifica otro.

frozen

Los valores de los atributos no pueden cambiar una vez inicializados, ni pueden añadirse o eliminarse valores. No se pueden añadir enlaces ni modificar los existentes tras la inicialización del objeto del extremo opuesto de la asociación (esto es, el extremo contrario a aquel que tiene el valor congelado). Sin embargo, cuando se crea un objeto en el extremo opuesto se pueden añadir enlaces al extremo congelado como parte de la inicialización.

addOnly

Se pueden añadir valores de atributos adicionales si la multiplicidad no es un número fijo o ya tiene el valor máximo. Tras la creación, no se pueden borrar ni modificar los valores mientras vive el objeto. Se pueden añadir nuevos enlaces pero no se pueden eliminar ni modificar después de su creación. Si se destruye un objeto participante, los objetos que lo contienen son borrados, a pesar de su status **addOnly**.

vértice

Origen o destino de una transición en una máquina de estados. Un vértice puede ser bien un estado o un pseudoestado.

visibilidad

Una enumeración cuyo valor (**pública**, **protegida** o **privada**) denota si el elemento del modelo al que alude se puede o no ver fuera del espacio de nombres que lo contiene.

Véase también acceso para una discusión de las reglas de visibilidad en su aplicación a referencias entre paquetes.

Semántica

La visibilidad declara la capacidad de un elemento de modelado para hacer referencia a un elemento que se encuentra en un espacio de nombres distinto de aquel al que pertenece el elemento que hace la referencia. La visibilidad es parte de la relación existente entre un elemento y el contenedor que lo contiene. El contenedor puede ser un paquete, clase o algún otro espacio de nombres. Existen tres visibilidades predefinidas.

public	Todo elemento que pueda ver el contenedor puede ver también el elemento indicado.
protected	Sólo pueden ver el elemento indicado los elementos pertenecientes a su contenedor o a un descendiente de su contenedor.
private	Sólo pueden ver el elemento indicado los elementos pertenecientes a ese mismo contenedor. Los demás elementos, incluyendo los pertenecientes a descendientes del contenedor, no pueden hacer referencia a él ni usarlo en modo alguno.

Se podrían definir otras clases de visibilidad para algunos lenguajes de programación, tales como la visibilidad *de implementación* propia de C++ (en realidad, todas las formas de visibilidad que no sea pública dependen del lenguaje). El uso de opciones adicionales tiene que hacerse por convenio entre el usuario y las posibles herramientas de modelado y generadores de código.

Notación

La visibilidad se puede mostrar mediante una palabra reservada de propiedad o bien mediante un signo de puntuación situado delante del nombre de algún elemento del modelo.

public	+
protected	#
private	—

Se puede suprimir el marcador de visibilidad. La ausencia de un marcador de visibilidad indica que no se muestra la visibilidad, no que sea pública o no esté definida. Las herramientas deberían asignar visibilidades a los nuevos elementos, aun cuando no se muestre la visibilidad. El marcador de visibilidad es una notación taquigráfica de la cadena completa de especificación de la propiedad de visibilidad.

También se puede especificar la visibilidad mediante palabras clave (**public**, **protected**, **private**). Esta forma suele emplearse como elemento en medio de una lista, aplicándose a todo un bloque de atributos u otros elementos de la lista.

Toda opción de visibilidad que sea propia de un lenguaje o definida por el usuario deberá ser especificada mediante una cadena de propiedad o bien mediante alguna convención propia de una herramienta.

Clases. En las clases el marcador de visibilidad se sitúa en una lista de elementos, tales como atributos y operaciones. Muestra si otras clases pueden o no acceder a esos elementos.

Asociaciones. En una asociación, el marcador de visibilidad se sitúa en el nombre de rol de la clase destino (el extremo al que se accedería empleando esa especificación de visibilidad). Muestra si la clase del extremo opuesto puede o no recorrer la asociación en la dirección del extremo que tiene asociado el marcador de visibilidad.

Paquetes. En un paquete, el marcador de visibilidad se sitúa en elementos contenidos directamente dentro del paquete, tales como clases, asociaciones y paquetes anidados. Muestra si otro paquete que acceda al primero o lo importe puede o no ver los elementos.

vista

Proyección de un modelo, que se ve desde una cierta perspectiva o punto de observación y emite aquellas entidades que no son relevantes para esa perspectiva. Aquí, la palabra no se usa para denotar un elemento de presentación. Se refiere más bien a las proyecciones tanto en el modelo semántico como en la notación visual.

vista de actividades

Aspecto del sistema que tiene que ver con la especificación del comportamiento como actividades conectadas por flujos de control. Esta vista contiene grafos de actividades y se representa mediante diagramas de actividades. De forma un tanto libre se agrupa con otras vistas que tienen que ver con el comportamiento para formar la vista dinámica.

Véase grafo de actividades.

vista de administración del modelo

Dícese de aquel aspecto de un modelo que trata de la organización del modelo en sí en partes estructuradas, a saber, paquetes, subsistemas y modelos. La vista de administración del modelo se considera a veces parte de la vista estática y suele combinarse con la vista estática en los diagramas de clases.

vista de casos de uso

Aspecto del sistema dedicado a la especificación de comportamiento en términos de casos de uso. Un modelo de casos de uso es un modelo que se centra en esta visualización. La vista de casos de uso es parte del conjunto de conceptos de modelado que están relativamente agrupados como vista dinámica.

vista de comportamiento

Vista del modelo que pone de relieve el comportamiento de las instancias existentes en un sistema, incluyendo sus métodos, colaboraciones e historias de estados.

vista de despliegue

Una vista que muestra los nodos en un sistema distribuido, los componentes que se almacenan en cada nodo, y los objetos que se almacenan en componentes y nodos.

Véase despliegue, diagrama de despliegue.

vista de implementación

Es una vista de un modelo que contiene una declaración estática de los componentes de un sistema, de sus dependencias y posiblemente de las clases que sean implementadas por ese componente.

Véase diagrama de componentes.

vista de interacción

Se trata de la visualización de un modelo que muestra el intercambio de mensajes entre objetos para lograr algún propósito. Consta de colaboraciones e interacciones y se muestra empleando diagramas de colaboración y diagramas de secuencia.

vista de máquina de estados

Aspecto del sistema relacionado con la especificación del comportamiento de elementos individuales a lo largo de su vida. Esta vista contiene máquinas de estados. Está relativamente agrupada con otras vistas de comportamiento en la vista dinámica.

vista dinámica

Ese aspecto de un modelo que se ocupa de la especificación y de la implementación del comportamiento a lo largo del tiempo, contrapuesto a la estructura estática que se encuentra en la vista estática. La vista dinámica es un término de agrupación que incluye la vista de casos de uso, la vista de máquina de estados, la vista de actividades, y la vista de interacción.

vista estática

Vista del modelo global que caracteriza los elementos de un sistema y sus relaciones entre sí. Incluye a los clasificadores y sus relaciones: sucesos, generalización, dependencia y realización. A veces recibe el nombre de *vista de clases*.

Semántica

La vista estática muestra la estructura estática de un sistema y en particular las clases de elementos que existen (tales como las clases y tipos), su estructura interna y sus relaciones con otros elementos. Las vistas estáticas no muestran información temporal, aunque puede contener apariciones materializadas de cosas que tengan o describan un comportamiento temporal, tal como las especificaciones de operaciones o de eventos.

Entre los componentes de más alto nivel de una vista estática se cuentan los clasificadores (clase, interfaz, tipo de datos), las relaciones (asociación, generalización, dependencia, reali-

zación), las restricciones y los comentarios. También contiene paquetes y subsistemas como unidades organizativas. Los demás componentes están subordinados y contenidos en los elementos de alto nivel.

Las vistas de implementación, de despliegue y de administración del modelo están relacionadas con la vista estática y suelen combinarse con ella en otros diagramas.

La vista estática se puede contrastar con la vista dinámica, que la complementa y se basa en ella.

vista estructural

Vista global de un modelo que hace hincapié en la estructura de los objetos del sistema, incluyendo sus tipos, clases, relaciones, atributos y operaciones.

vista funcional

Una vista que se ocupa de la descomposición de un sistema en las funciones o las operaciones que proporcionan su funcionalidad. Una vista funcional no se considera normalmente orientada a objetos y puede conducir a una arquitectura difícil de mantener.

En los métodos de desarrollo tradicionales, el diagrama de flujo de datos es el corazón de la vista funcional. UML no apoya directamente una visión funcional, aunque los grafos de actividades tienen algunas características funcionales.

Los elementos estándar son palabras clave predefinidas que corresponden a restricciones, estereotipos y etiquetas. Representan conceptos de utilidad general que no son suficientemente significativos o suficientemente diferentes de los conceptos centrales para incluirlos como conceptos centrales de UML. Tienen la misma relación con los conceptos centrales de UML que la que posee una biblioteca de subrutinas con un lenguaje de programación. No forman parte del lenguaje en sí, pero forman parte del entorno con que puede contar el usuario cuando hace uso del lenguaje. La lista incluye también palabras clave de notación —son palabras reservadas que aparecen en el símbolo de otro elemento del modelo pero que denotan elementos incorporados del modelo, no estereotipos—. Para las palabras clave se enumera el símbolo de la notación.

Las referencias cruzadas aluden a artículos del Capítulo 13, Enciclopedia de Términos.

access

(estereotipo de la dependencia Permiso)

Dependencia estereotipada entre dos paquetes que denota que el contenido público del paquete destino es visible para el espacio de nombres del paquete origen.

Véase acceso.

association

(estereotipo de ExtremoAsociación)

Restricción aplicada al extremo de una asociación (incluyendo los extremos de un enlace o un extremo de un rol de asociación), que especifica que la instancia correspondiente es visible a través de una asociación virtual y no a través de un enlace transitorio, tal como un parámetro o variable local.

Véase asociación, extremo de asociación, rol de asociación.

become

(estereotipo de la relación Flujo)

Dependencia estereotipada cuyo origen y destino representan la misma instancia en distintos instantes de tiempo, pero pudiendo tener cada una valores, instancia de estado y roles diferentes.

Una dependencia de conversión que va desde **A** hasta **B** significa que la instancia **A** se convierte en **B** con (posiblemente) nuevos valores, instancia de estado y roles. La notación de conversión es una flecha discontinua que va del origen al destino con la palabra clave «**become**».

Véase become en la enciclopedia de términos.

bind

(palabra reservada en un símbolo de dependencia)

Palabra reservada de dependencia que denota una relación de ligadora. Va seguida por una lista de argumentos separados por comas y encerrada entre paréntesis.

Véase enlace, elemento ligado, plantilla.

call

(estereotipo de la dependencia Uso)

Dependencia estereotipada cuyo origen es una operación y cuyo destino es una operación. Una dependencia de llamada específica que el origen invoca a la operación destino. Una dependencia de llamada puede conectar una operación origen con cualquier operación destino situada a su alcance, incluyendo (pero sin tener como límite) las operaciones del clasificador que la contiene y las operaciones de otros clasificadores visibles.

Véase llamada, uso.

complete

(restricción de Generalización)

Restricción que se aplica a un conjunto de generalizaciones, para especificar que se han especificado todos los hijos (aunque se haya omitido alguno) y que no se espera declarar hijos adicionales posteriormente.

Véase generalización.

copy

(estereotipo de la relación Flujo)

Relación de flujo estereotipada cuyo origen y destino son instancias distintas, pero teniendo ambas los mismos valores, instancia de estado y roles (aunque con distinta identidad). Una dependencia de copia de **A** hacia **B** significa que **B** es una copia exacta de **A**. Los futuros

cambios de A no se reflejan necesariamente en B. La notación de copia es una flecha discontinua que va del origen al destino con la palabra clave «**copy**».

Véase acceder, copia.

create

(estereotipo de característica de comportamiento)

Característica de comportamiento estereotipada que denota que la característica indicada crea una instancia del clasificador al que está asociada la característica.

(estereotipo de evento)

Evento estereotipado que denota que la instancia que encierra la máquina de estados a que se aplica el evento se crea. La creación se puede aplicar únicamente a una transición inicial situada en el nivel más elevado de la máquina de estados. De hecho, éste es el único tipo de disparador que se puede aplicar a una transición inicial.

(estereotipo de la dependencia Utilización)

Dependencia estereotipada que denota que el clasificador cliente crea instancias del clasificador proveedor.

Véase creación, uso.

derive

(estereotipo de la dependencia Abstracción)

Dependencia estereotipada cuyo origen y destino suelen ser (aunque no necesariamente) elementos del mismo tipo. Una dependencia de derivación específica que el origen se puede computar a partir del destino. El origen puede haberse implementado por razones de diseño, tales como la eficiencia, aun cuando sea redundante desde un punto de vista lógico.

Véase derivación, elemento derivado.

destroy

(estereotipo de característica de comportamiento)

Característica de comportamiento estereotipada que denota que la característica indicada destruye una instancia del clasificador al que está asociada la característica.

(estereotipo de evento)

Evento estereotipado que denota la destrucción de la instancia que contiene a la máquina de estados a la que se aplica ese tipo de evento.

Véase destrucción.

destroyed

(restricción de Rol de clasificador y de Rol de asociación)

Denota que existe una instancia de ese rol al principio de la ejecución de la interacción que lo contiene pero esa instancia es destruida antes de finalizar la ejecución.

Véase rol de asociación, rol de clasificador, colaboración, destrucción.

disjoint

(restricción de la Generalización)

Restricción que se aplica a un conjunto de generalizaciones, especificando que un objeto no puede ser instancia de más de un hijo en ese conjunto de generalizaciones. Esta situación únicamente surgiría con herencia múltiple.

Véase generalización.

document

(estereotipo de componente)

Componente estereotipado que representa un documento.

Véase componente.

documentation

(etiqueta de un elemento)

Comentario, descripción o explicación del elemento al que está asociada.

Véase comentario, cadena.

enumeration

(palabra clave de los símbolos de clasificador)

Palabra reservada de un tipo de datos de enumeración, cuyos detalles especifican un dominio que consta de un conjunto de identificadores que son los posibles valores de una instancia de ese tipo de datos.

Véase enumeración.

executable

(estereotipo de componente)

Componente estereotipado que denota un programa que se puede ejecutar en un nodo.

Véase componente.

extend

(palabra reservada del símbolo de Dependencia)

Palabra clave de un símbolo de dependencia que denota una relación de extensión entre casos de uso.

Véase extender.

facade

(estereotipo de paquete)

Paquete estereotipado que contiene únicamente referencias a elementos del modelo que son propiedad de otro paquete. Se emplea para proporcionar una vista pública de cierta parte del contenido de un paquete. Una fachada no contiene ningún elemento del modelo que sea propio de ella.

Véase paquete.

file

(estereotipo de componente)

Archivo es un componente estereotipado que representa un documento que contiene código fuente o bien datos.

Véase componente.

framework

(estereotipo de Paquete)

Paquete estereotipado que consta principalmente de patrones.

Véase paquete.

friend

(estereotipo de dependencia de permiso)

Dependencia estereotipada cuyo origen es un elemento del modelo tal como una operación, clase o paquete y cuyo destino es un elemento diferente del modelo, tal como una clase o

paquete. La relación de amistad concede al destino acceder al origen, independientemente de la visibilidad declarada. Extiende la visibilidad del origen de tal modo que el destino puede ver el interior del origen.

Véase acceso, amigo, visibilidad.

global

(estereotipo de ExtremoAsociacion)

Restricción que se aplica a un extremo de asociación (incluyendo los extremos de enlace y los extremos de roles de asociación), especificando que el objeto asociado es visible porque se encuentra en un alcance global respecto al objeto que está en el otro extremo del enlace.

Véase asociación, extremo de asociación, colaboración.

implementation

(estereotipo de generalización)

Generalización estereotipada que denota que el cliente hereda la lista implementación del proveedor (sus atributos, operaciones y métodos) pero no hace públicas las interfaces del proveedor ni garantiza que va a admitirlas, violando por tanto la capacidad de sustitución. Es una herencia privada.

Véase generalización, herencia privada.

implementationClass

(estereotipo de clase)

Clase estereotipada que no es un tipo y que representa una implementación de una clase en algún lenguaje de programación. Un objeto puede ser instancia de un máximo de una clase de implementación. Por contraste, un objeto puede ser una instancia de múltiples clases ordinarias en un instante y perder o ganar clases con el paso del tiempo. Una instancia de una clase de implementación también puede ser instancia de cero o más tipos.

Véase clase de implementación, tipo.

implicit

(estereotipo de asociación)

Estereotipo de una asociación, que especifica que la asociación no es manifiesta (no está implementada) sino sólo conceptual.

Véase asociación.

import

(estereotipo de dependencia de permiso)

Dependencia estereotipada entre dos paquetes, que denota que el contenido público del paquete destino se añade al espacio de nombres del paquete origen.

Véase acceso, importar.

include

(palabra clave de símbolo de dependencia)

Palabra clave de los símbolos de dependencia que denota una relación de inclusión entre casos de uso.

Véase incluir.

incomplete

(restricción de generalización)

Restricción que se aplica a un conjunto de generalizaciones, especificando que no se han especificado todos los hijos y que se espera la adición de otros hijos.

Véase generalización.

instanceOf

(palabra clave de símbolo de dependencia)

Una metarelación cuyo cliente es una instancia y cuyo proveedor es un clasificador. Una dependencia **instanceOf** que va desde **A** hacia **B** indica que **A** es una instancia de **B**. La notación de **instanceOf** es una flecha discontinua con la palabra clave «**instanceOf**».

Véase descriptor, instancia, instancia de.

instantiate

(estereotipo de dependencia de uso)

Dependencia estereotipada entre clasificadores, que indica que las operaciones que afectan al cliente crean instancias del proveedor.

Véase instanciación, uso.

invariant

(estereotipo de restricción)

Restricción estereotipada que es preciso asociar a un conjunto de clasificadores o relaciones. Denota las condiciones que debe imponer la restricción a efectos de los clasificadores o relaciones y sus instancias.

Véase invariante.

leaf

(palabra reservada de ElementoGeneralizable y CaracterísticaDeComportamiento)

Indica un elemento que no puede tener descendientes o no se pueden anular, esto es, que no es polimórfico.

Véase hoja, polimórfico/a.

library

(estereotipo de componente)

Componente estereotipado que representa una biblioteca estática o dinámica.

Véase componente.

local

(estereotipo de ExtremoAsociación)

Estereotipo de un extremo de asociación, extremo de enlace o extremo de rol de asociación, que especifica que el objeto asociado se encuentra en el alcance local del objeto que está en el otro extremo.

Véase asociación, extremo de asociación, colaboración, enlace transitorio.

location

(etiqueta de símbolo clasificador)

El componente que sirve de base al clasificador.

(palabra reservada de símbolo de instancia de componente)

Instancia de nodo en la que reside la instancia de componente.

Véase componente, localización, nodo.

metaclass

(estereotipo de clasificador)

Clasificador estereotipado que denota que la clase es una metaclass de alguna otra clase.

Véase metaclass.

new

(restricción de RolClasificador y Roldeasociación)

Denota que se crea una instancia del rol durante la ejecución de la interacción que lo contiene y que esa instancia sigue existiendo al finalizar la ejecución.

Véase rol de asociación, rol de clasificador, colaboración, creación.

overlapping

(restricción de generalización)

Restricción aplicada a un conjunto de generalizaciones, que especifica que un objeto puede ser instancia de más de un hijo dentro de ese conjunto de generalizaciones. Sólo puede surgir esta situación con herencia múltiple o clasificación múltiple.

Véase generalización.

parameter

(estereotipo de ExtremoAsociación)

Estereotipo de un extremo de asociación (incluyendo el extremo de un enlace y el extremo de un rol de asociación), que especifica que un objeto asociado es un argumento de una llamada a una operación del objeto situado en el otro extremo.

Véase rol de asociación, rol de clasificador, colaboración, parámetro, enlace transitorio.

persistence

(etiqueta de clasificador, asociación y atributo)

Denota si el valor de una instancia debe o no sobrevivir al proceso que lo ha creado. Los valores son persistentes y transitorios. Si se usa en un atributo, permite una mejor discriminación acerca de cuáles de los valores de los atributos deberían mantenerse dentro de un clasificador.

Véase objeto persistente.

postcondition

(estereotipo de restricción)

Restricción estereotipada que debe estar asociada a una operación. Denota las condiciones que deben cumplirse después de haber invocado a la operación.

Véase postcondición.

powertype

(estereotipo de clasificador)

Clasificador estereotipado que denota que el clasificador es una metaclassa cuyas instancias son subclases de otra clase.

(palabra reservada de símbolo de dependencia)

Relación cuyo cliente es un conjunto de generalizaciones y cuyo proveedor es un supratipo. El proveedor es un supratipo del cliente.

Véase supratipo.

precondition

(estereotipo de restricción)

Restricción estereotipada que debe estar asociada a una operación. Denota las condiciones que deben cumplirse en el momento de invocar a la operación.

Véase precondition.

process

(estereotipo de clasificador)

Clasificador estereotipado que es una clase activa que representa a un proceso pesado.

Véase clase activa, proceso, hilo.

refine

(estereotipo de dependencia de abstracción)

Estereotipo de dependencia que denota una relación de refinamiento.

Véase refinamiento.

requirement

(estereotipo de comentario)

Comentario estereotipado que enuncia una responsabilidad u obligación.

Véase requisito, responsabilidad.

responsibility

(estereotipo de comentario)

Contrato u obligación del clasificador. Se expresa mediante una cadena de texto.

Véase responsabilidad.

self

(estereotipo de ExtremoAsociación)

Estereotipo de un extremo de asociación (incluyendo los extremos de enlace y los extremos de rol de asociación), que especifica un pseudoenlace de un objeto consigo mismo con el propósito de invocar a una operación del mismo objeto dentro de una interacción. No implica una estructura de datos real.

Véase rol de asociación, rol de clasificador, colaboración, parámetro, enlace transitorio.

semantics

(etiqueta de clasificador)

Especificación del significado del clasificador.

(etiqueta de operación)

Especificación del significado de la operación.

Véase semántica.

send

(estereotipo de dependencia de uso)

Dependencia estereotipada cuyo cliente es una operación o clasificador y cuyo proveedor es una señal, que especifica que el cliente envía la señal a algún destino no especificado.

Véase enviar, señal.

stereotype

(palabra clave de símbolo clasificador)

Palabra clave para la definición de un estereotipo. Se puede utilizar el nombre como nombre de estereotipo en otros elementos del modelo.

Véase estereotipo.

stub

(estereotipo de paquete)

Paquete estereotipado que representa un paquete que proporciona las partes públicas de otro paquete, pero nada más.

Observe que la palabra también se usa en UML para describir las transiciones abreviadas.

Véase paquete.

system

(estereotipo de paquete)

Paquete estereotipado que contiene un conjunto de modelos de un sistema, describiéndolo desde distintos puntos de vista no necesariamente disjuntos —es la estructura de más alto nivel en la especificación del sistema—. También contiene relaciones entre elementos del modelo pertenecientes a diferentes modelos. Estas relaciones y restricciones no añaden información semántica a los modelos. Lo que hacen es describir las relaciones de los modelos en sí; por ejemplo para el seguimiento de requisitos y de la historia del desarrollo.

Un sistema puede ser realizado por un conjunto de sistemas subordinados, cada uno de los cuales es descrito por su propio conjunto de modelos reunidos en un paquete de sistema por separado. Un paquete de sistema sólo puede estar contenido en un paquete de sistema.

Véase paquete, modelo, sistema.

table

(estereotipo de componente)

Componente estereotipado que representa una tabla de una base de datos.

Véase componente.

thread

(estereotipo de clasificador)

Clasificador estereotipado que es una clase activa y representa un flujo de control ligero.

Observe que en este libro se utiliza la palabra con un sentido más amplio, denotando cualquier centro de ejecución independiente y concurrente.

Véase clase activa, hilo.

trace

(palabra clave de dependencia de abstracción)

Palabra clave de un símbolo de dependencia que denota una relación de traza.

Véase traza.

transient

(restricción de rol de clasificador y rol de asociación)

Afirma que se crea una instancia del rol durante la ejecución de la interacción que lo contiene, pero esa instancia se destruye antes de haber finalizado la ejecución.

Véase rol de asociación, rol de clasificador, colaboración, creación, destrucción, enlace transitorio.

type

(estereotipo de clase)

Clase estereotipada que se emplea para la especificación de un dominio de instancias (objetos) junto con las operaciones que son aplicables a esos objetos. Un tipo no puede contener métodos pero puede poseer atributos y asociaciones.

Véase clase de implementación, tipo.

use

(palabra clave de símbolo de dependencia)

Palabra clave de dependencia que denota una relación de uso.

Véase uso.

utility

(estereotipo de clasificador)

Clasificador estereotipado que carece de instancias. Describe una colección con nombre de atributos y operaciones que no son miembros, todos los cuales tienen alcance de clase.

Véase utilidad.

xor

(restricción de asociación)

Restricción aplicada a un conjunto de asociación que comparten una conexión con una clase; especifica que todo objeto de la clase compartida tendrá enlaces procedentes de una sola de las asociaciones. Se trata de una restricción o-exclusivo (no o-inclusivo).

Véase asociación.

Parte 4: Apéndices



Apéndice A

Metamodelo de UML



Documentos de definición de UML

UML está definido en un conjunto de documentos publicados por el Object Management Group [UML-98]. Estos documentos se han incluido en el CD que acompaña al libro. Este capítulo explica la estructura del modelo semántico de UML que se describe en los documentos.

El UML se define formalmente empleando un metamodelo —esto es, un modelo de las estructuras de UML. El metamodelo, a su vez, se expresa en UML. Esto es un ejemplo de intérprete metacircular— esto es, un lenguaje definido en términos de sí mismo. Pero las cosas no son completamente circulares. Sólo se emplea un pequeño subconjunto de UML para definir el metamodelo. En principio, el punto fijo de la definición se podría basar en alguna definición más básica. En la práctica, no es necesario llegar a estos extremos.

Cada sección del documento semántico contiene un diagrama de clases que muestra una porción del metamodelo; una descripción textual de las clases del metamodelo definidas en esa sección, con sus atributos y relaciones; una lista de restricciones aplicables a los elementos expresadas en lenguaje natural y en OCL y por último una descripción textual de la semántica dinámica de las estructuras de UML definidas en la sección. Por tanto, la semántica dinámica es informal, pero una descripción completamente formal sería a la vez poco práctica y prácticamente ilegible para casi todos.

La notación se describe en otro capítulo que hace referencia al capítulo de semántica y establece la correspondencia entre los símbolos y las clases del metamodelo.

Estructura del metamodelo

El metamodelo está dividido en tres paquetes principales (Figura A1).

- El paquete de fundamentos define la estructura estática de UML.
- El paquete de elementos de comportamiento define la estructura dinámica de UML.
- El paquete de administración del modelo define la estructura organizativa de los modelos de UML.

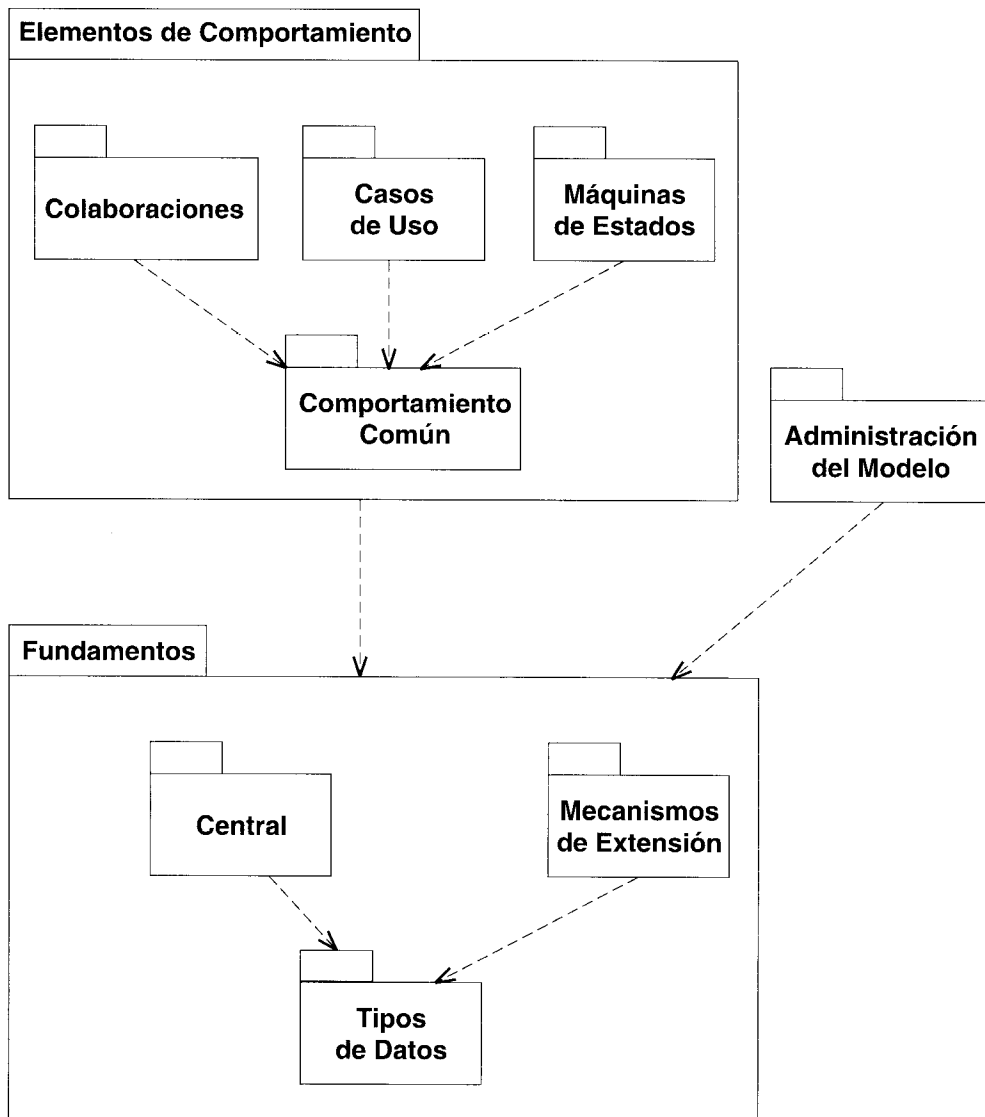


Figura A-1 Estructura de paquetes del metamodelo de UML

Paquete de fundamentos

El paquete de fundamentos contiene cuatro fundamentos.

Núcleo

El paquete núcleo describe las principales estructuras estáticas de UML. Entre ellas se cuentan los clasificadores, su contenido y sus relaciones. El contenido incluye atributo, operación, método y parámetro. Entre las relaciones se cuentan generalización, asociación y dependencia. También se definen varias metaclases abstractas tales como elemento generalizable, espacio de

nombres y elemento del modelo. El paquete también define plantilla y distintos tipos de subclases de dependencia así como componente, nodo y comentario.

Tipos de datos

El paquete de tipos de datos describe las clases de tipos de datos que se utilizan en el meta-modelo.

Mecanismos de extensión

El paquete de mecanismos de extensión describe los mecanismos restricción, estereotipo y valor etiquetado.

Paquete de elementos de comportamiento

El paquete de comportamiento tiene un subpaquete para cada vista principal y además un paquete para estructuras de comportamiento que comparten las tres vistas principales.

Comportamiento común

El paquete de comportamiento común describe señal, operación y acción. También describe instancias de clases correspondientes a diferentes descriptores.

Colaboraciones

El paquete colaboraciones describe colaboración, interacción, mensaje, rol de clasificador y asociación.

Casos de uso

El paquete casos de uso describe actor y caso de uso.

Máquinas de estados

El paquete máquinas de estados describe la estructura de una máquina de estados, incluyendo estado y distintas clases de pseudoestado, evento, señal, transición y condición de guarda. También describe estructuras adicionales para modelos de actividad tales como estado de acción, estado de actividad y estado de flujo de objeto.

Paquete de administración de modelos

El paquete de administración de modelos describe los paquetes, modelos y subsistemas. También describe las propiedades de propiedad y visibilidad de los espacios de nombres y de los paquetes. No posee subpaquetes.

Apéndice B

Resumen de la notación



Este capítulo contiene un breve resumen visual de la notación. Se han incluido los elementos principales de la notación, pero no constan todas las versiones u opciones. Para estudiar todos los detalles, consulte la entrada de cada elemento en la enciclopedia.

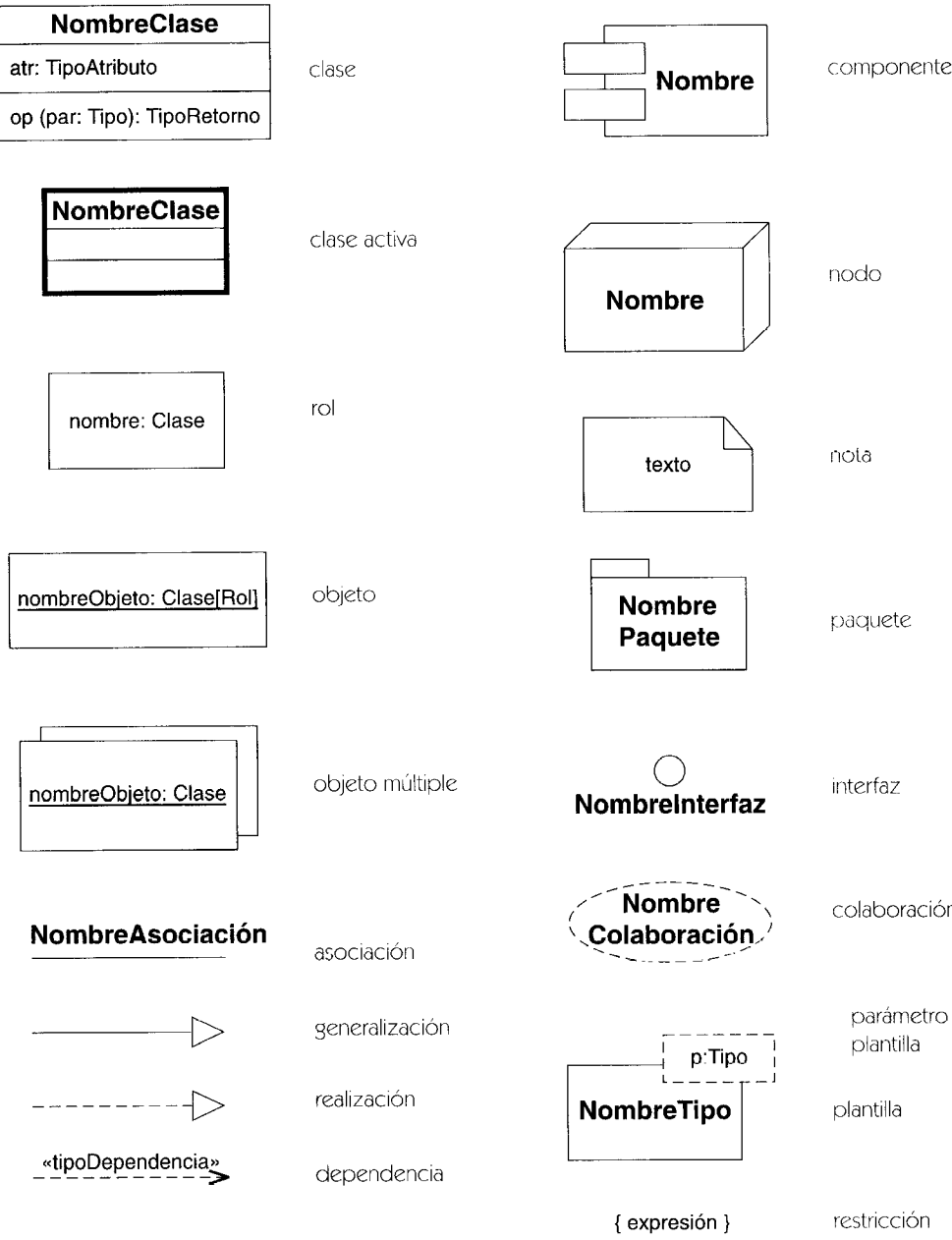


Figura B-1 Iconos de los diagramas de clase, componente, despliegue y colaboración

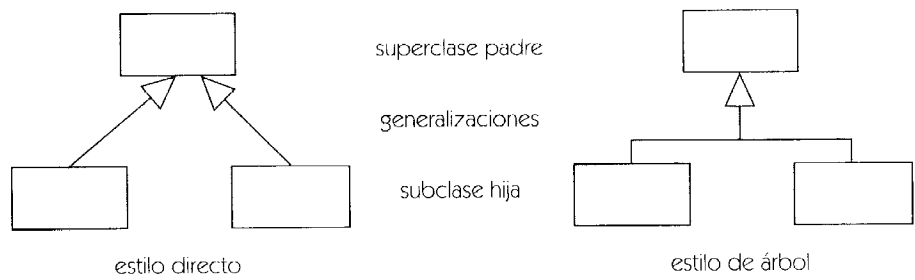


Figura B-4 Generalización

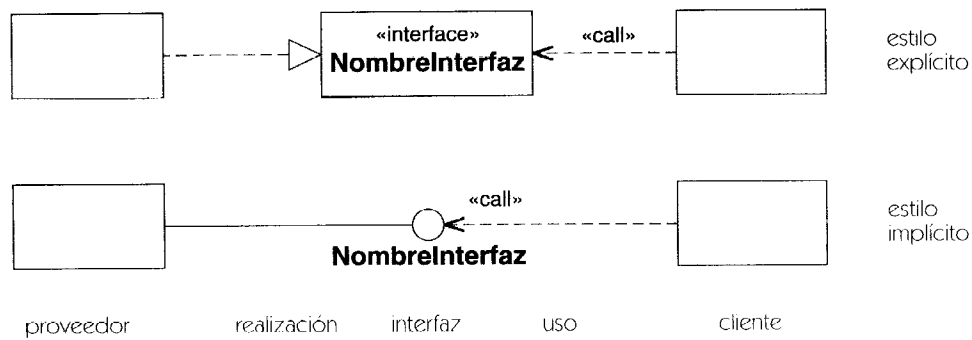


Figura B-5 Realización de una interfaz

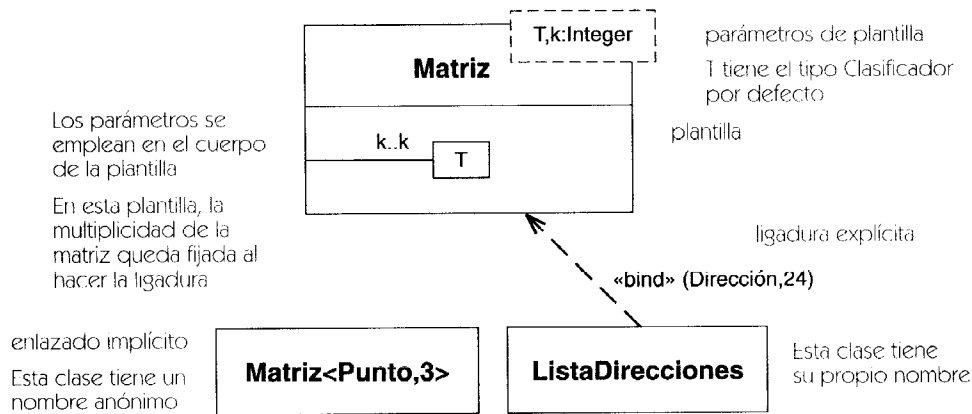
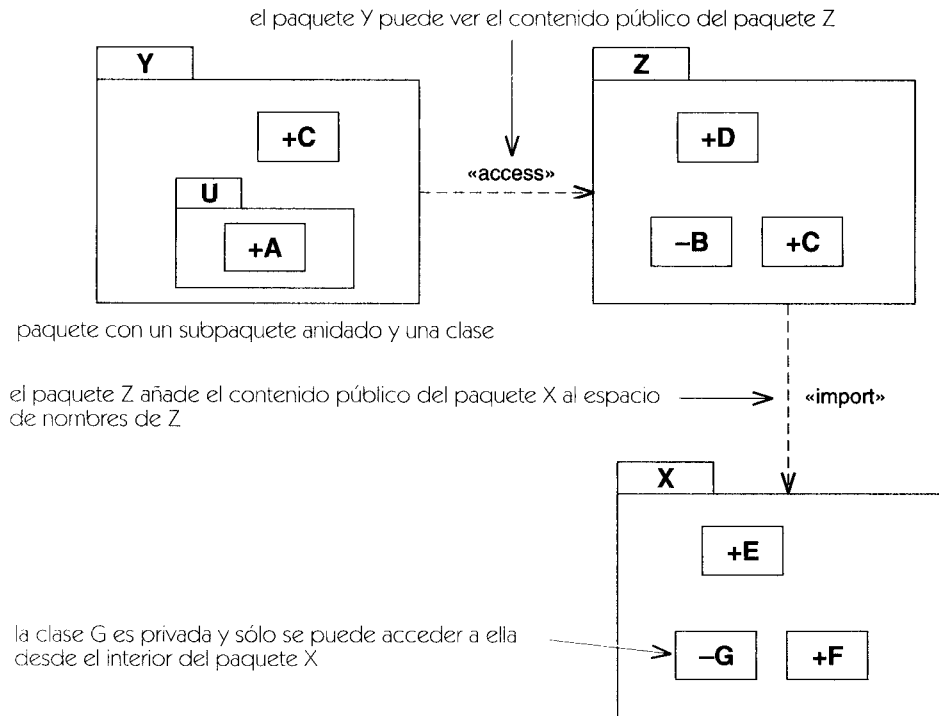
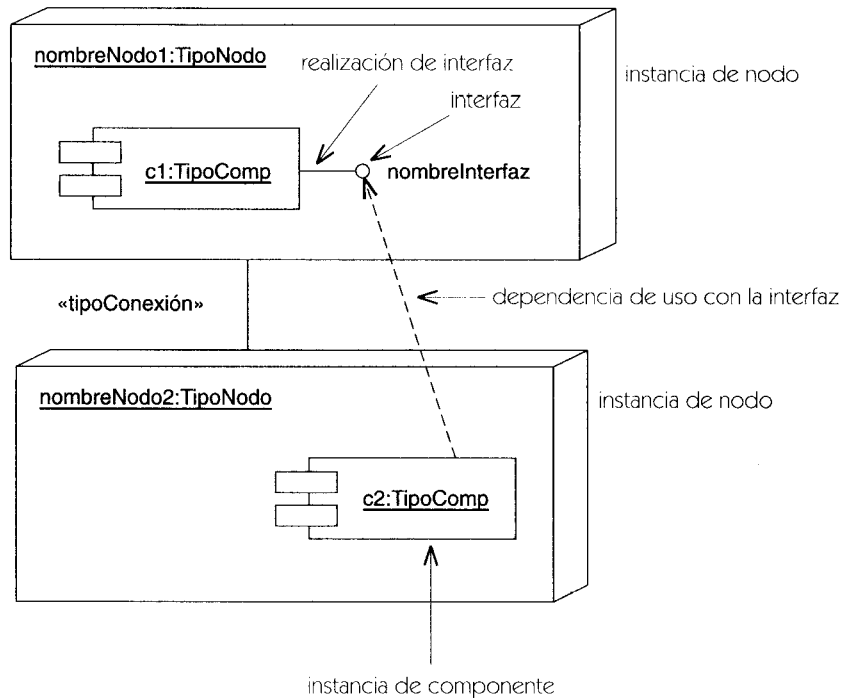


Figura B-6 Plantilla

**Figura B-7** Notación de paquetes**Figura B-8** Notación de componentes y nodos

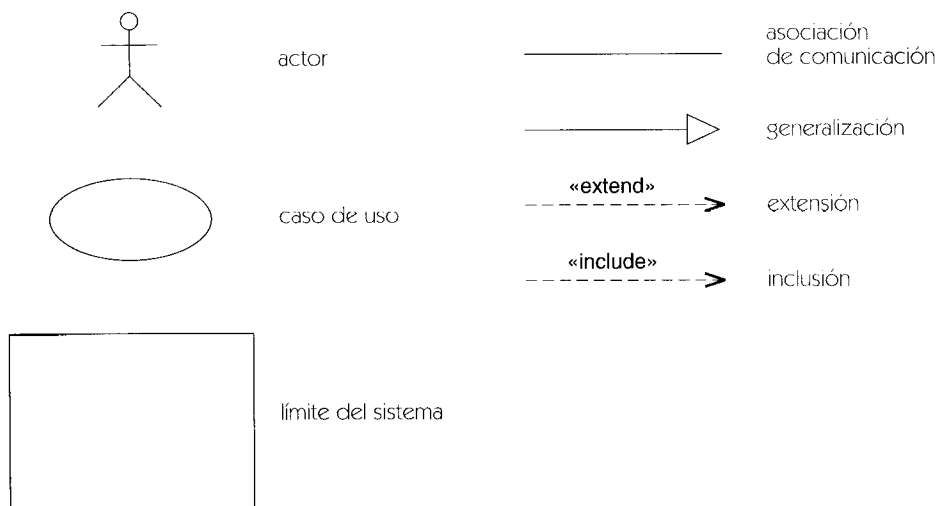


Figura B-9 Iconos de los diagramas de caso de uso

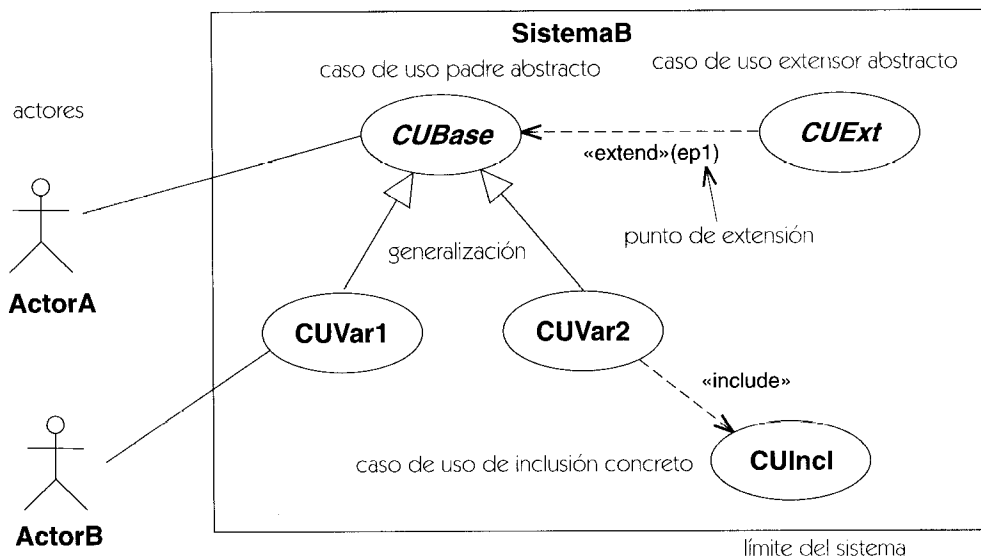


Figura B-10 Notación de los diagramas de caso de uso

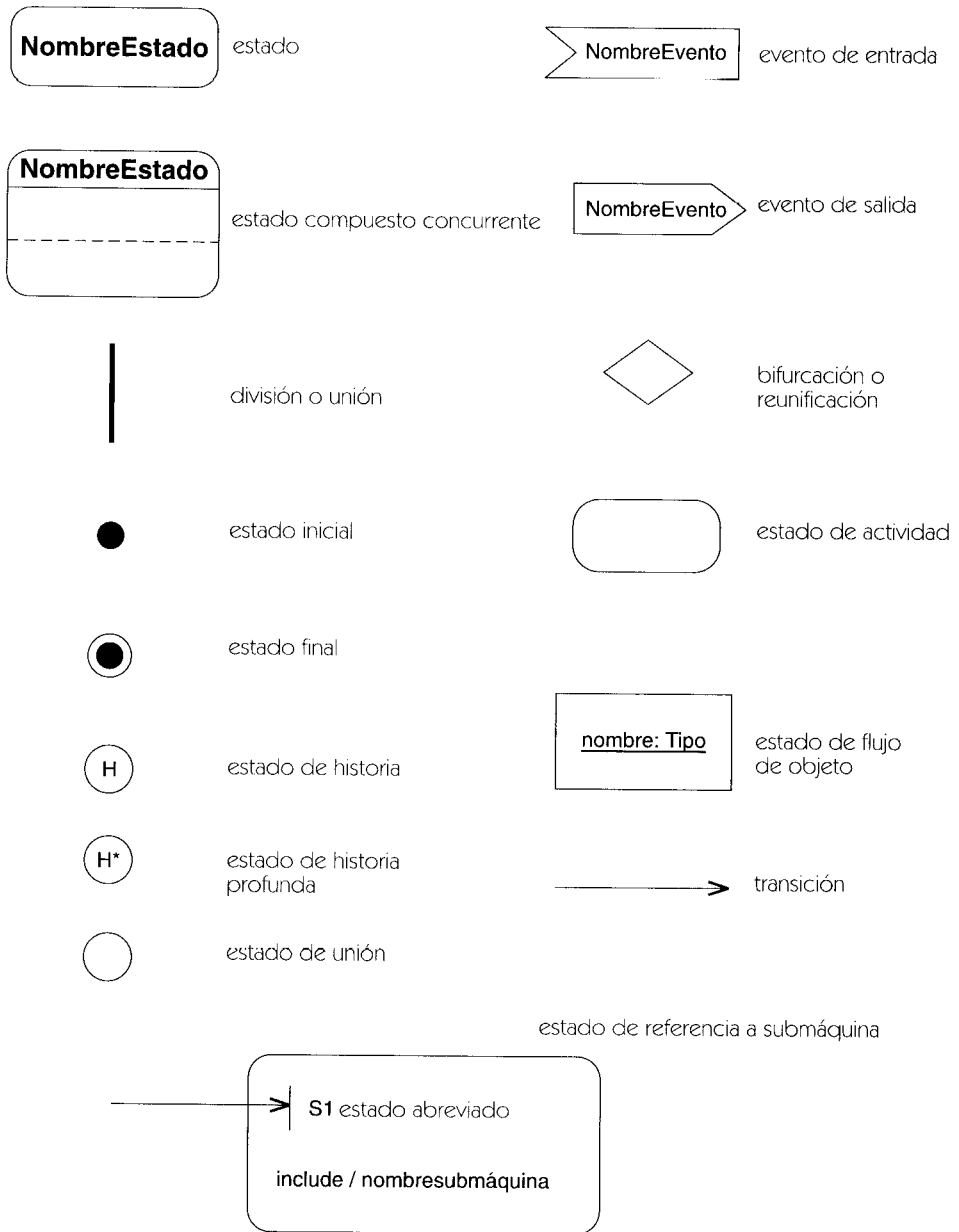


Figura B-11 Iconos de un diagrama de estados y diagramas de actividad

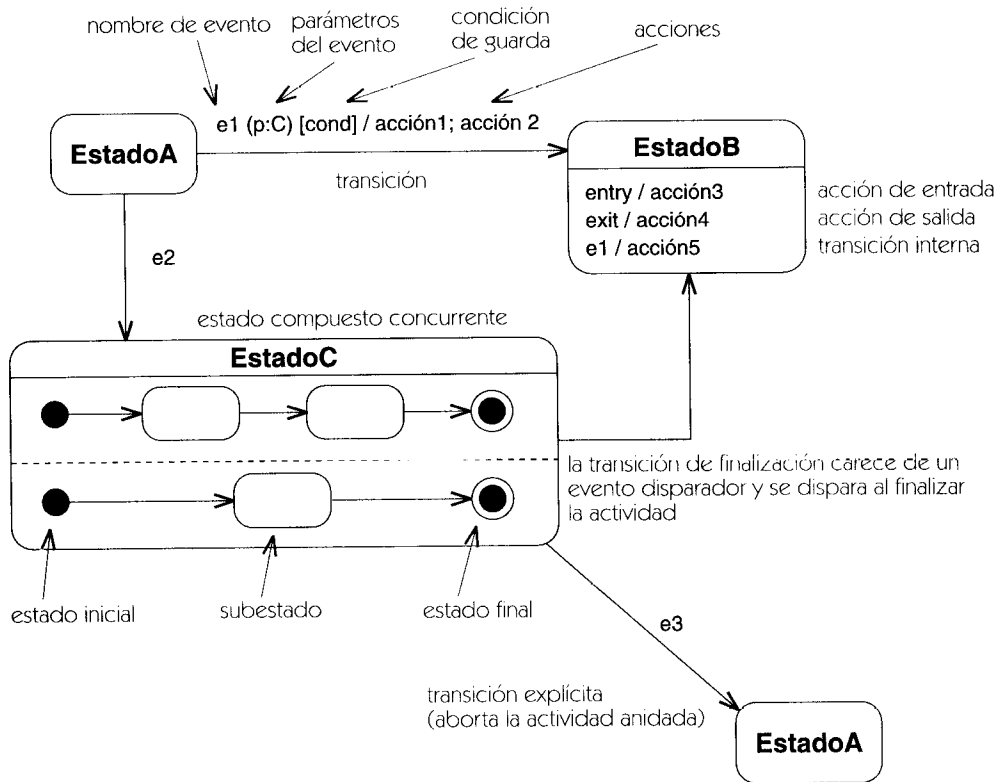
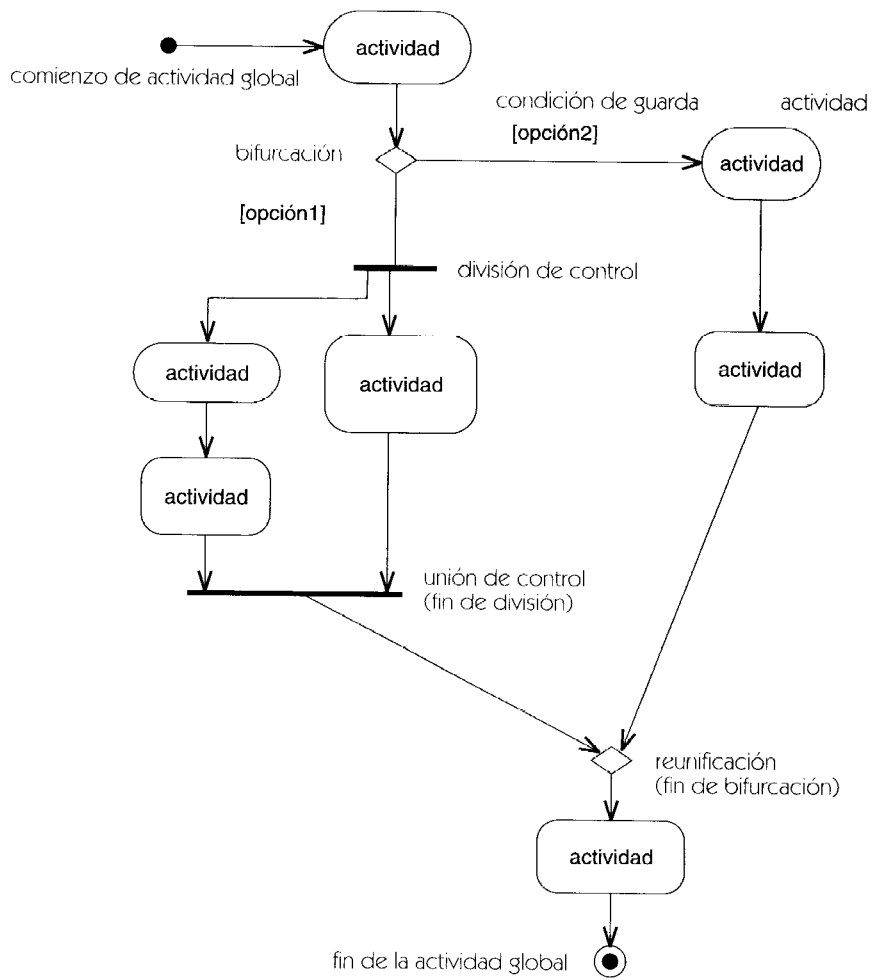


Figura B-12 Notación de diagramas de estados

NombreClase::Nombreoperación**Figura B-13** Notación de los diagramas de actividad

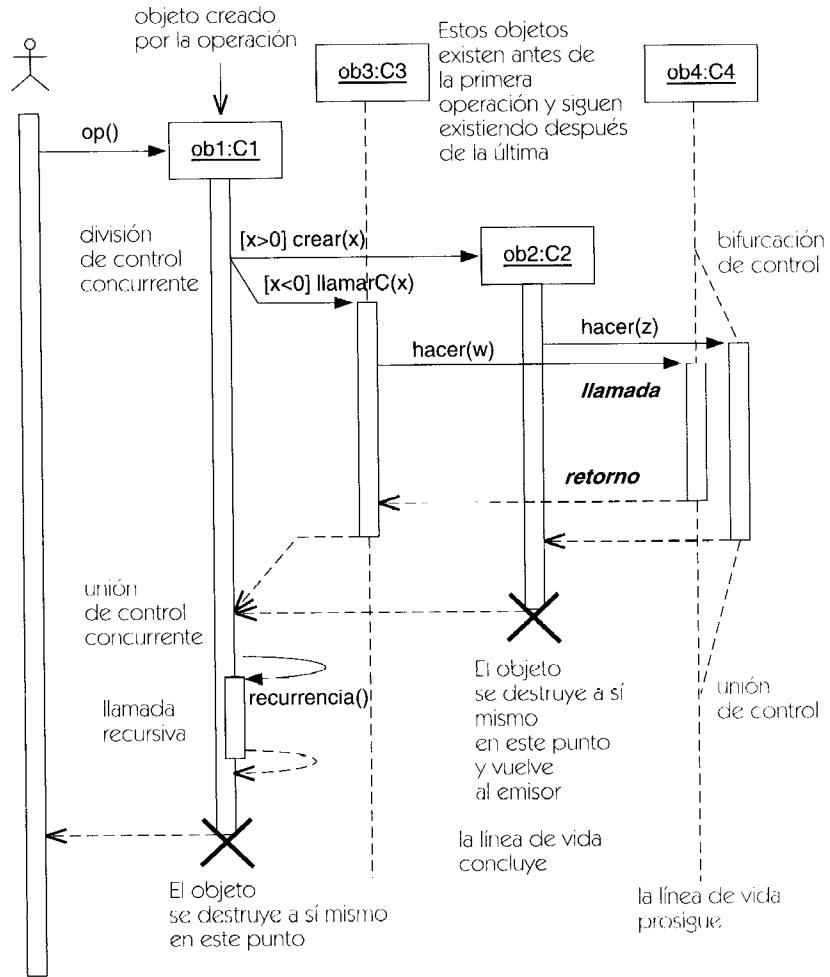


Figura B-14 Notación de diagramas de secuencia

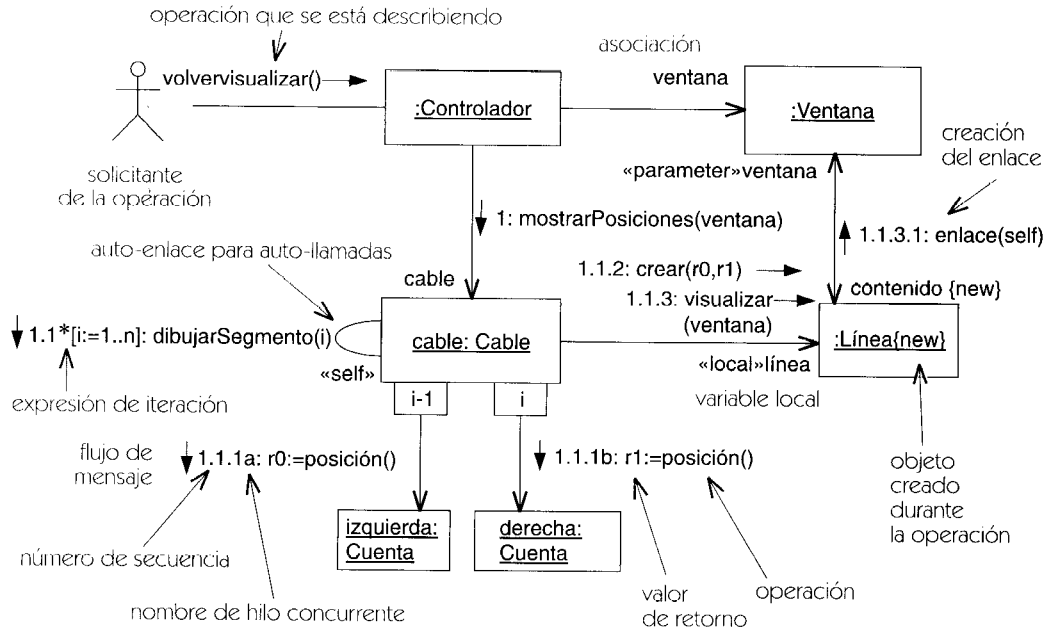


Figura B-15 Notación de diagrama de colaboración

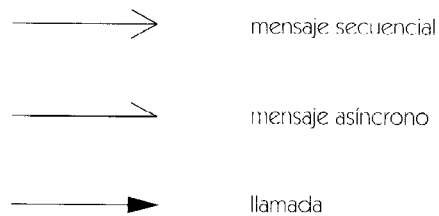


Figura B-16 Notación de mensajes

Apéndice C

Extensiones del proceso



Cómo personalizar UML

UML está diseñado para ser utilizable con múltiples propósitos. Aunque UML es un lenguaje universal que puede expresar un elevado número de propiedades fundamentales de modelado, existen ocasiones en que resulta deseable una jerga hecha a la medida de un determinado dominio de conocimiento. En los lenguajes naturales, la jerga, las germanías y los vocabularios especializados suelen facilitar la comunicación dentro de un arte o disciplina especializado, pero impiden la comunicación con los extraños. UML se puede personalizar de forma similar, empleando distintos mecanismos tales como convenciones de denominación, reglas de estilo, clases predefinidas y correspondencias implícitas con el software. En particular, se puede definir una extensión de UML empleando estereotipos, restricciones y valores etiquetados predefinidos.

UML posee muchas estructuras expresivas así que debería evitarse una extensión especializada a no ser que sea absolutamente necesaria. Por definición, una extensión sirve tan sólo a una comunidad limitada y puede dar lugar a malentendidos y confusiones por parte de los extraños. Sin embargo, si una comunidad destino está muy centrada, la potencia expresiva de una extensión puede compensar a veces la pérdida de uniformidad. Con el tiempo, algunas extensiones pueden ser tan útiles que acabarán por ser añadidas a UML; otras pueden caer en desuso.

Este capítulo describe dos extensiones que se adjuntan con los documentos de UML. Su utilización no es obligatoria y ni siquiera se recomienda necesariamente, pero indican la forma en que se puede definir una extensión para un proceso de desarrollo. En la literatura se ha descrito un cierto número de extensiones similares.

Extensiones del proceso de desarrollo de software

Estas extensiones están basadas en el proceso de desarrollo Objectory, que es un precursor del Proceso Unificado. Están destinadas a ser utilizadas en un proceso de desarrollo de software que consta de cuatro fases: captura de casos de uso, análisis, diseño e implementación.

Estas extensiones incluyen estereotipo de unidades de empaquetado, clases y asociación así como algunas restricciones relativas a la conexión de elementos. No hay etiquetas nuevas.

Estereotipos organizativos

La Tabla C-1 muestra los estereotipos organizativos, esto es, los estereotipos de modelo, clase y subsistema. Hay varios estereotipos aplicables a cada fase del desarrollo. En el proceso Objectory, cada fase posee un modelo distinto, con relaciones de traza entre los elementos de los diferentes modelos. Avanzando desde arriba hacia abajo, los términos sistema, subsistema y paquete de servicio describen capas de empaquetado.

Los estereotipos organizativos no tienen iconos especiales. Se representan mediante iconos de carpeta con el nombre del estereotipo entre comillas, tal y como aparece en la Figura C-1.

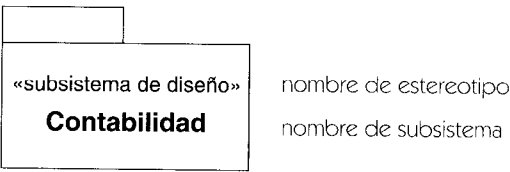


Figura C-1 Notación organizativa de estereotipos para el proceso de desarrollo de software

Tabla C-1 Estereotipos organizativos para el proceso de desarrollo de software

Clase Base	Estereotipos por Fase del Proceso			
	Caso de Uso	Análisis	Diseño	Implementación
modelo	modelo de caso de uso	modelo de análisis	modelo de diseño	modelo de implementación
paquete	sistema de caso de uso paquete de caso de uso			sistema de implementación subsistema de implementación
subsistema		sistema de análisis subsistema de análisis paquete de servicio	sistema de diseño subsistema de diseño paquete de servicio	

Estereotipos de clase

En las clases están definidos tres estereotipos: control, frontera y entidad. Se muestran en la Figura C-2.

Una *clase de control* describe objetos que administran interacciones, tales como un administrador de transiciones, un controlador de dispositivos o un monitor de un sistema operativo.

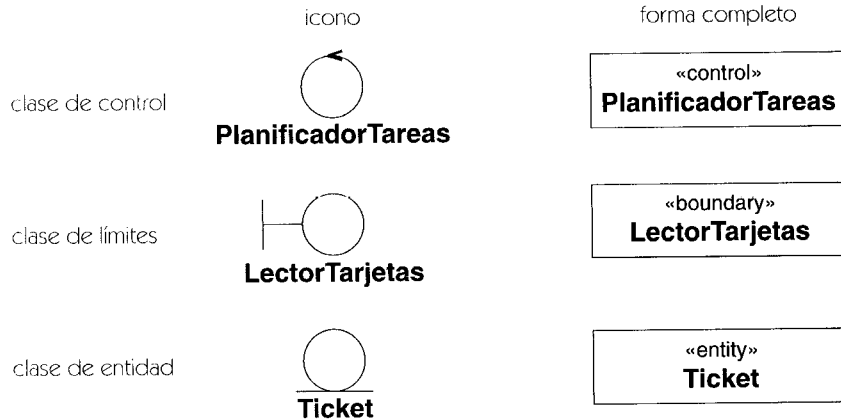


Figura C-2 Estereotipo de clase para el proceso de desarrollo de software

Tiene un comportamiento específico de un caso de uso. Normalmente, las clases de control no sobreviven a los casos de uso que admiten. Las clases de control se muestran como un círculo que tiene una punta de flecha.

Las *clases de frontera* describen objetos que median entre el sistema y los actores externos, tales como un formulario para hacer pedidos o un sensor. Se muestran como un círculo que tiene asociado un segmento en forma de T. Los objetos de frontera suelen existir durante toda la vida del sistema.

Las *clases de entidad* describen objetos pasivos. No inician las interacciones. Los objetos de entidad pueden participar en muchos casos de uso y normalmente sobreviven a las transiciones individuales. Se muestran como un círculo con una línea debajo.

Estereotipos de asociación

Existen dos estereotipos de asociación: comunicar y suscribir. La notación es una ruta de asociación con el nombre del estereotipo entre comillas.

Las *asociaciones de comunicación* conectan un actor con un caso de uso con el cual se comunica. Ésta es la única asociación posible entre actores y casos de uso, así que se puede omitir la palabra clave.

Las *asociaciones de suscripción* conectan la clase cliente (el suscriptor) con la clase proveedora (el editor). El suscriptor especifica un conjunto de eventos que pueden ser producidos por el editor. Cuando se produce uno de estos eventos, se le notifica al suscriptor.

Extensiones de modelado de negocios

Estas extensiones están destinadas a modelar organizaciones de negocios del mundo real y para comprender situaciones reales, más que para la implementación de software. Estos estereotipos no son necesarios para el modelado de negocios, pero abarcan algunas situaciones comunes.

Estas extensiones incluyen estereotipos de unidades de empaquetado, clases y asociaciones así como algunas restricciones relativas a la conexión de elementos. No hay etiquetas nuevas.

Estereotipos organizativos

La Tabla C-2 muestra los estereotipos organizativos correspondientes al modelado de procesos de negocios.

El modelo de caso de uso es un modelo de la vista de casos de uso. El sistema de casos de uso y el paquete de casos de uso son dos capas de organización de su contenido.

El modelo de objetos es un modelo de la estructura interna del sistema de negocios. El sistema objeto es su subsistema de más alto nivel, que contiene unidades organizativas y unidades de trabajo como capas inferiores. Una unidad organizativa corresponde a una unidad organizativa del negocio real, y una unidad de trabajo es un agrupamiento significativo, aun siendo de menor entidad.

No existen iconos especiales para la unidades organizativas del modelado de negocios; utilizan el símbolo de carpeta con un estereotipo entre comillas.

Tabla C-2 Estereotipos organizativos para el modelado de negocios

<i>Clase base</i>	<i>Captura de casos de uso</i>	<i>Modelo de Objetos</i>
modelo	modelo de casos de uso	modelo de objetos
paquete	sistema de caso de uso paquete de caso de uso	
subsistema		sistema de objetos unidad organizativa unidad de trabajo

Estereotipos de clase

Además de los actores (que están definidos en UML estándar), los objetos de negocios tiene varios sistemas de clase: trabajador, trabajador de caso, trabajador interno, entidad. Se muestran en la Figura C-3.

Un trabajador representa un ser humano que actúa en el interior del sistema. Un trabajador de caso es un trabajador que interacciona directamente con actores externos. Un trabajador interno es un trabajador que interacciona con trabajadores y entidades situadas dentro del sistema.

Las entidades de clase describen objetos pasivos. No inician las interacciones. Los objetos de entidad pueden participar en muchos casos de uso y normalmente sobreviven a interacciones individuales. En una situación de trabajo, las entidades suelen representar productos del trabajo.

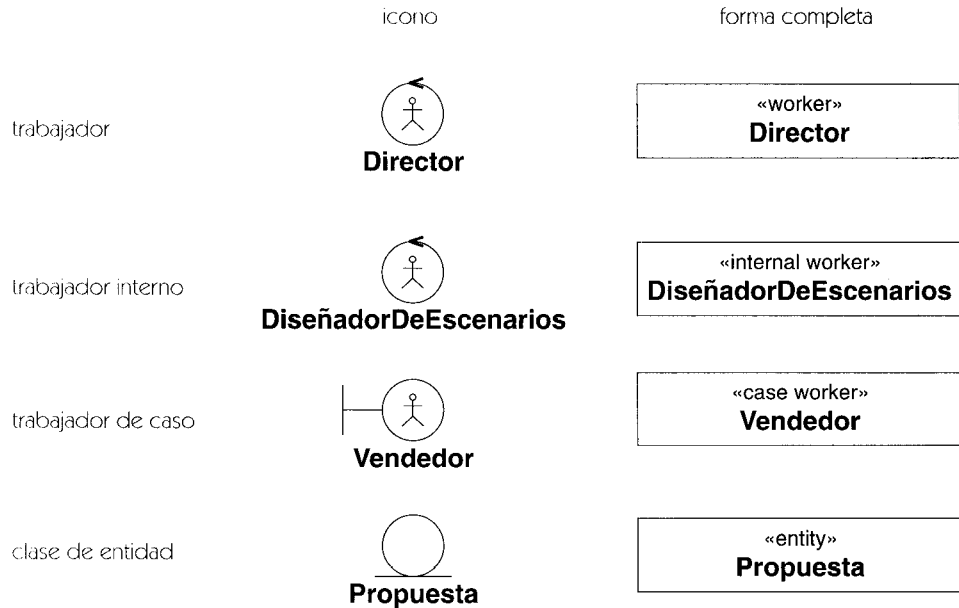


Figura C-3 Estereotipos de clase para el modelado de negocios

Estereotipos de asociación

Existen dos estereotipos de asociación: comunicar y suscribir. Son los mismos que para un proceso de desarrollo de software. La notación es una ruta de asociación con la palabra clave del estereotipo entre comillas.

Una asociación de comunicación conecta un actor con aquel caso de uso con el cual se comunica. Se trata de la única asociación existente entre actores y casos de uso, así que se puede omitir la palabra clave.

Una asociación de suscripción conecta una clase cliente (el suscriptor) con una clase proveedora (el editor). El suscriptor especifica un conjunto de eventos que puede producir el editor. Cuando se produce uno de estos eventos, se le comunica al suscriptor.

